CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Guia 5 - Integrar el servicio de notificaciones por email Brevo al microservicios user-management

Instrucciones: Lee atentamente la siguiente guía paso a paso para integrar el servicio de envío de correos de Brevo en tu microservicio. Asegúrate de seguir cada uno de los procedimientos descritos para la configuración local y el despliegue en la nube.

Objetivos de aprendizaje:

1. Comprender el funcionamiento de Brevo como proveedor de API RESTful y SMTP para envío transaccional de correos.
2. Configurar la autenticación segura, llaves de API y credenciales SMTP dentro del proyecto Spring Boot.
3. Implementar la tolerancia a fallos y resiliencia en clientes HTTP usando Resilience4j.
4. Desplegar la solución integrada en la plataforma cloud Render gestionando adecuadamente las variables de entorno.

Quinta parte: Servicio en la nube de envío masivo de correo electrónico mediante API RESTful

En esta quinta parte aprenderemos a crear y configurar una cuenta en Brevo, una plataforma en la nube especializada en el envío masivo y transaccional de correo electrónico. Brevo ofrece una capa gratuita ideal para proyectos de desarrollo y producción de menor escala, integrándose fácilmente con aplicaciones Java y Spring Boot mediante protocolo SMTP o API RESTful.
URL: https://www.brevo.com/

¿Qué es Brevo y por qué utilizarlo?

Brevo (anteriormente conocido como Sendinblue) es una plataforma integral de marketing y mensajería transaccional. En el desarrollo de software, se utiliza principalmente como servidor SMTP externo o proveedor de API de correo para delegar la entrega de mensajes (como activación de cuentas, restablecimiento de contraseñas y notificaciones del sistema) sin necesidad de mantener un servidor de correo propio ni lidiar con bloqueos de spam por IP de servidor no verificada.

Creación de la cuenta en TiDB

Paso 1: Acceso al portal e inicio de registro
Para comenzar, accede al sitio web oficial de Brevo. En la página principal se destacan sus servicios en la nube y el cumplimiento de normativas de privacidad como el RGPD. Haz clic en el botón **Pruébalo gratis** para iniciar el proceso de creación de cuenta.

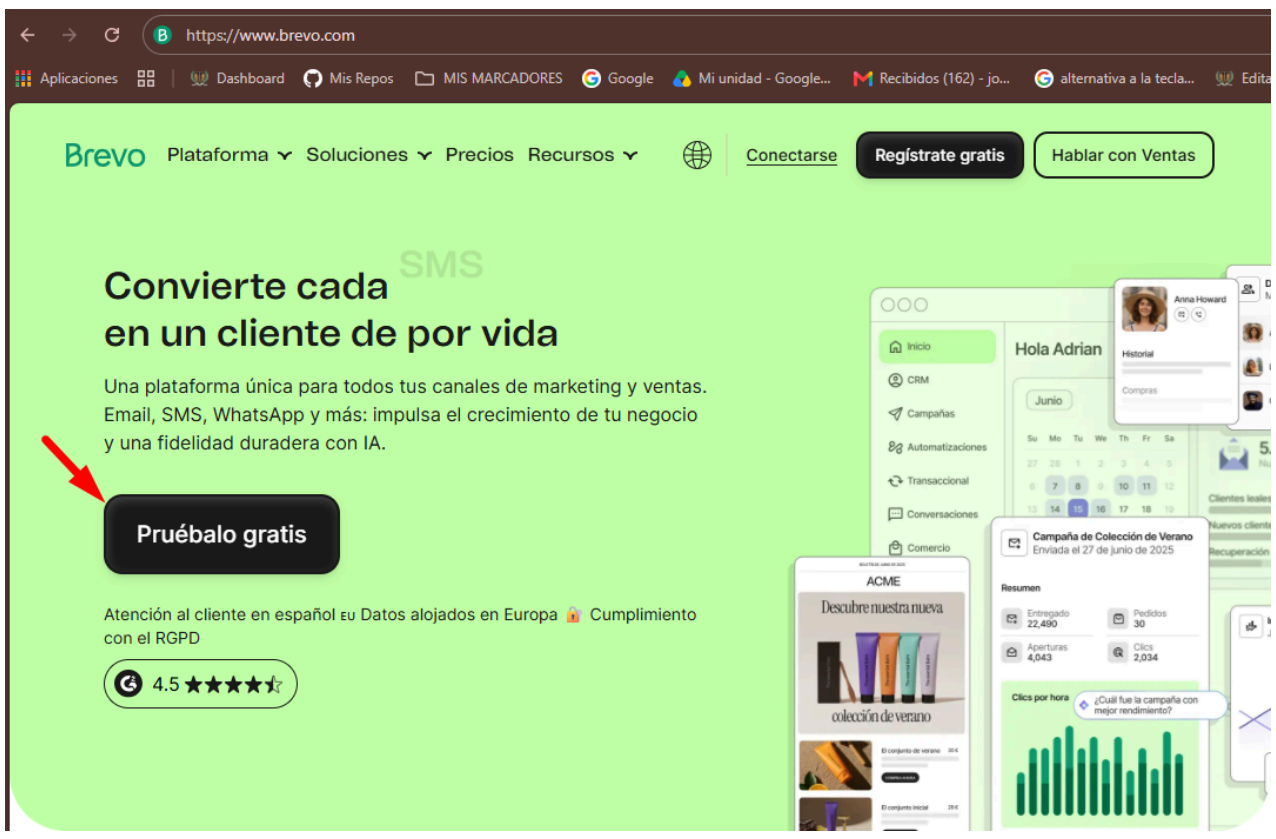


Imagen 1: Página principal de Brevo resaltando el botón Pruébalo gratis.

Paso 2: Selección del método de autenticación

Brevo ofrece múltiples opciones para registrarte: mediante credenciales tradicionales (correo y contraseña) o mediante proveedores de identidad federada (OAuth 2.0) como Google o Apple. Se recomienda la opción **Registrarse con Google** para simplificar el acceso y mantener un control centralizado de seguridad.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

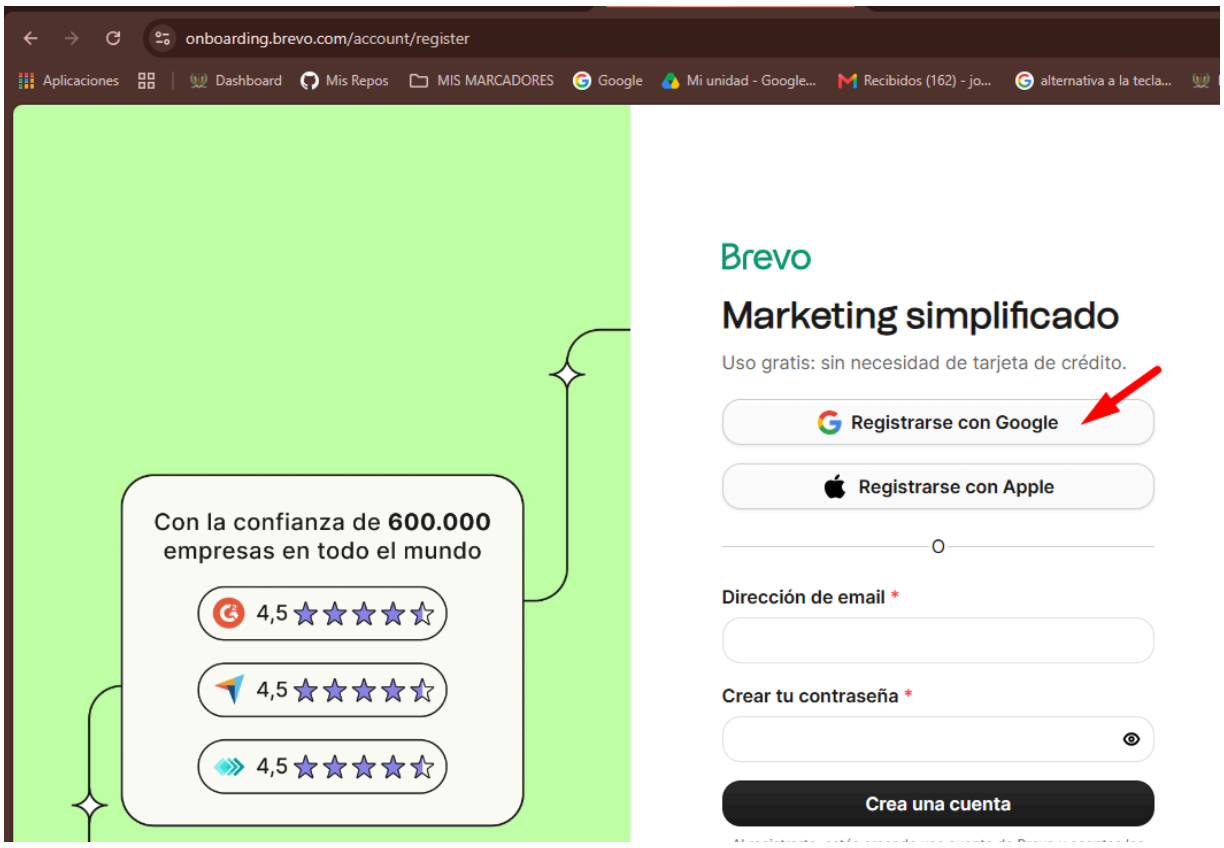


Imagen 2: Formulario de registro de Brevo mostrando la opción Registrarse con Google.

Paso 3: Selección de la cuenta de Google

Al seleccionar la autenticación con Google, el navegador desplegará la ventana emergente con las cuentas asociadas. Elige la cuenta institucional o personal con la que administrarás los servicios de correo de tu aplicación.

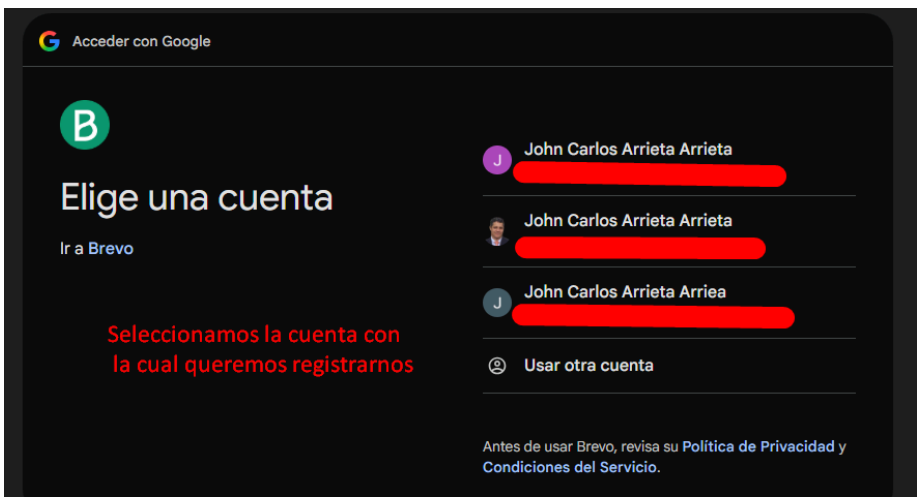


Imagen 3: Ventana de autenticación de Google para seleccionar la cuenta.

Paso 4: Confirmación de permisos de acceso

A continuación, Google te solicitará autorizar el intercambio de información básica de perfil (nombre, foto y dirección de correo electrónico) con la plataforma Brevo. Revisa las condiciones y haz clic en **Continuar**.

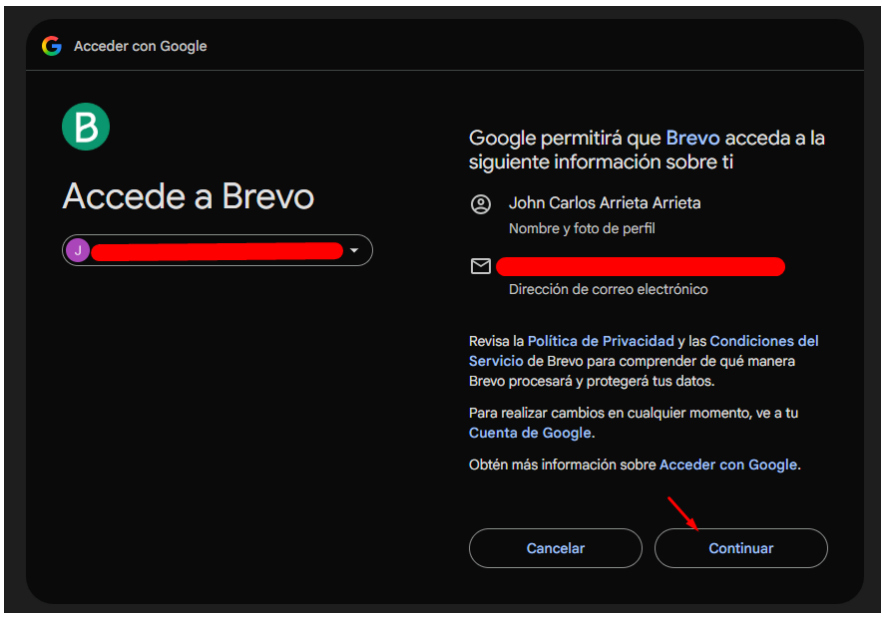


Imagen 4: Pantalla de confirmación de permisos de cuenta de Google para Brevo.

Paso 5: Registro de información de la organización

En la pantalla de bienvenida, completa el perfil del usuario introduciendo tus nombres, apellidos y el nombre de la organización o proyecto. Estos datos se emplean para personalizar la cuenta y verificar que los envíos cumplen con los estándares de identificación requeridos.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

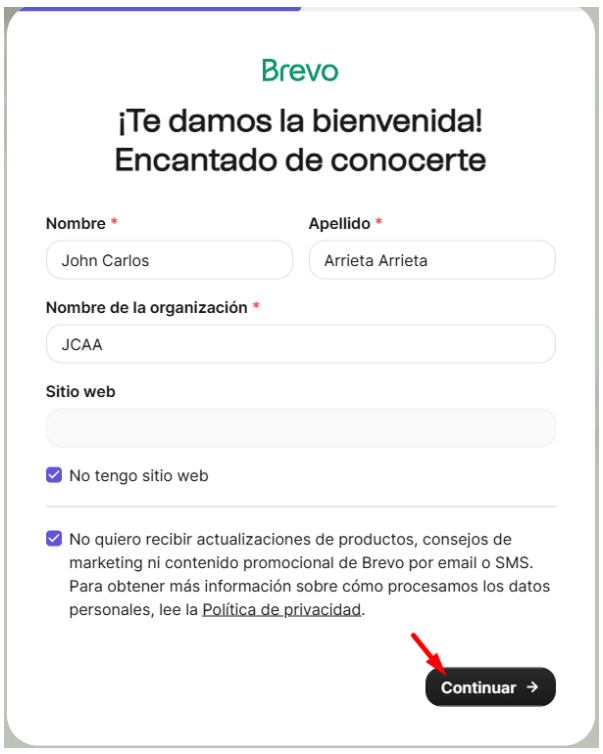


Imagen 5: Formulario de perfil del usuario u organización en Brevo.

Paso 6: Configuración de la dirección física (Cumplimiento Antispam)

Las regulaciones internacionales contra el spam (como la ley CAN-SPAM y el RGPD) exigen que todo correo saliente incluya la dirección física de la organización emisora. Completa los campos de dirección, código postal, ciudad y país, y luego haz clic en **Seleccionar tu plan**.



Imagen 6: Formulario de dirección postal requerida por normativas antispam.

Paso 7: Selección del plan de servicio

Selecciona el plan de suscripción que se ajuste a tus necesidades. Para entornos de desarrollo, aprendizaje e integraciones iniciales, selecciona el plan **Gratis**.



Imagen 7: Desplegable de selección del plan Gratis en Brevo.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 8: Verificación y análisis de características del plan gratuito

El plan gratuito de Brevo incluye una cuota de hasta 300 correos diarios sin costo alguno. Esta capacidad es más que suficiente para realizar pruebas unitarias, de integración y despliegues iniciales en aplicaciones desarrolladas con Spring Boot. Se recomienda revisar detenidamente el cuadro comparativo para confirmar que las funciones necesarias para tu entorno se encuentran totalmente activas.

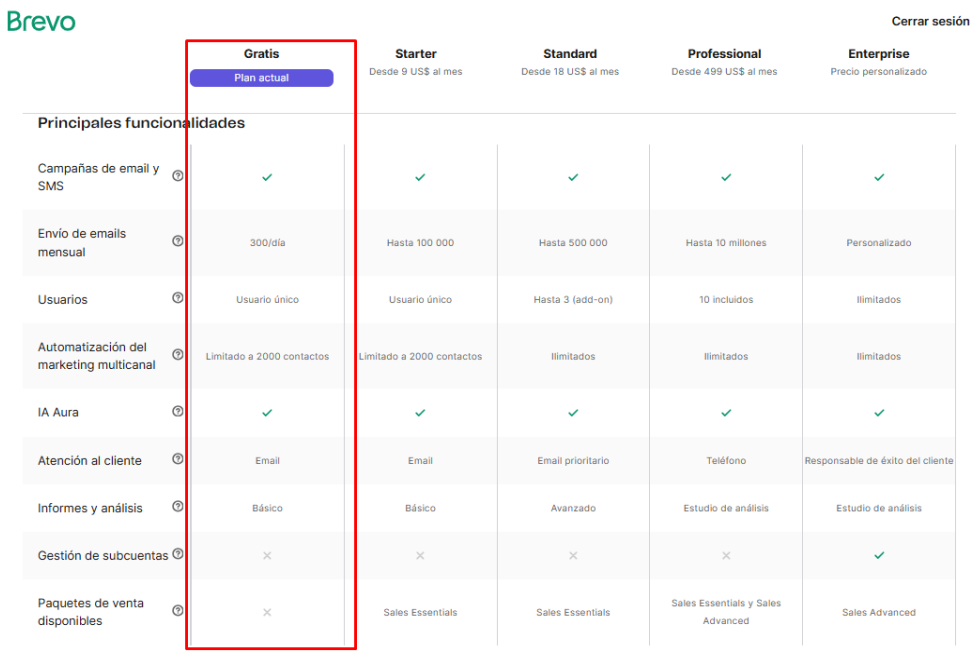


Imagen 8: Tabla comparativa de planes resaltando la columna del plan Gratis.

Una vez completado el proceso de suscripción, si notas que el panel administrativo no actualiza de manera automática las opciones del menú lateral, cierra la sesión e inicia nuevamente. Esto forzará la recarga completa de los módulos y privilegios administrativos asignados a tu nueva cuenta.

Paso 9: Acceso al menú de configuración principal

Desde el panel de control principal (Dashboard), dirígete a la esquina superior derecha y haz clic en el icono del engranaje (Configuración). Este menú desplegará las opciones avanzadas de administración, parámetros globales de la organización e integraciones técnicas.

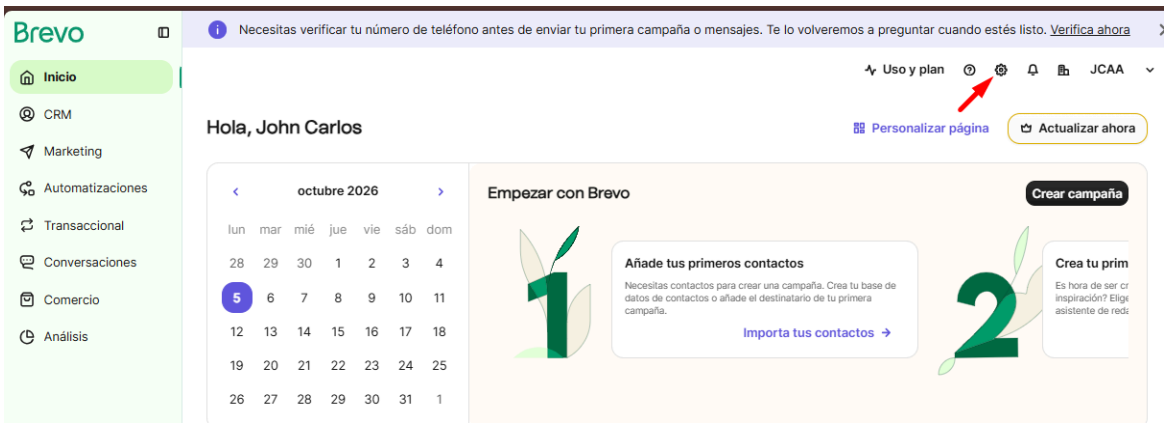


Imagen 9: Panel principal de Brevo indicando el botón de Configuración.

Paso 10: Navegación hacia la sección SMTP y API

En la barra lateral izquierda, ubica el bloque denominado **Configuración de la organización** y selecciona la opción **SMTP y API**. Este apartado centraliza los parámetros de conexión, credenciales de correo transaccional y llaves de acceso para la integración con sistemas externos.

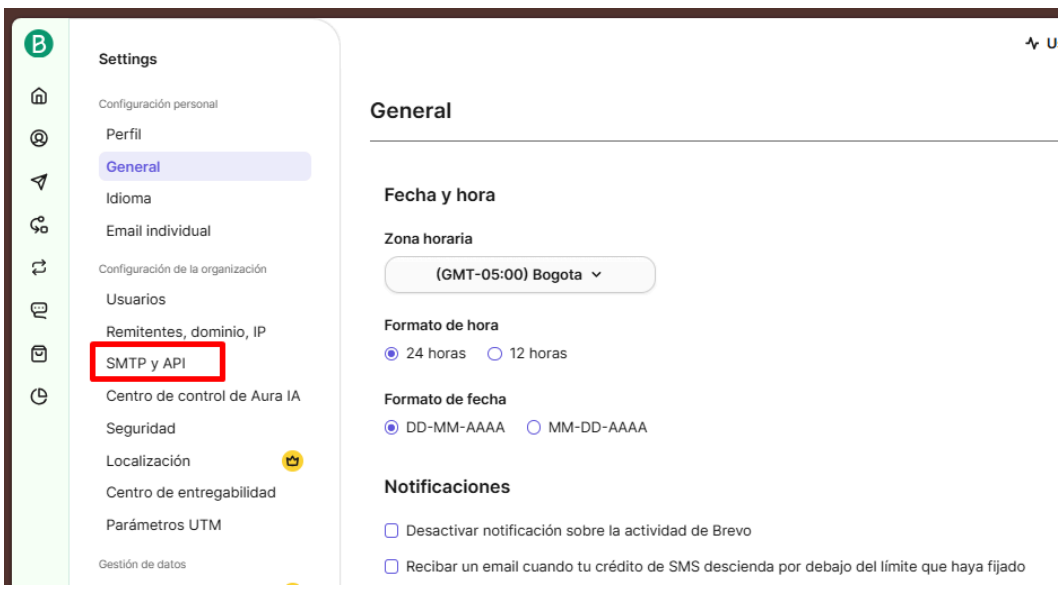


Imagen 10: Menú de configuración resaltando la opción SMTP y API.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 11: Inspección de parámetros SMTP y control de seguridad IP

En esta vista se detallan los datos de conexión del servidor SMTP saliente (Servidor: **`smtp-relay.brevo.com`**, Puerto: **`587`**, Usuario/Iniciar sesión). Para generar un nuevo secreto de acceso, haz clic en **+ Generar una nueva clave SMTP**.

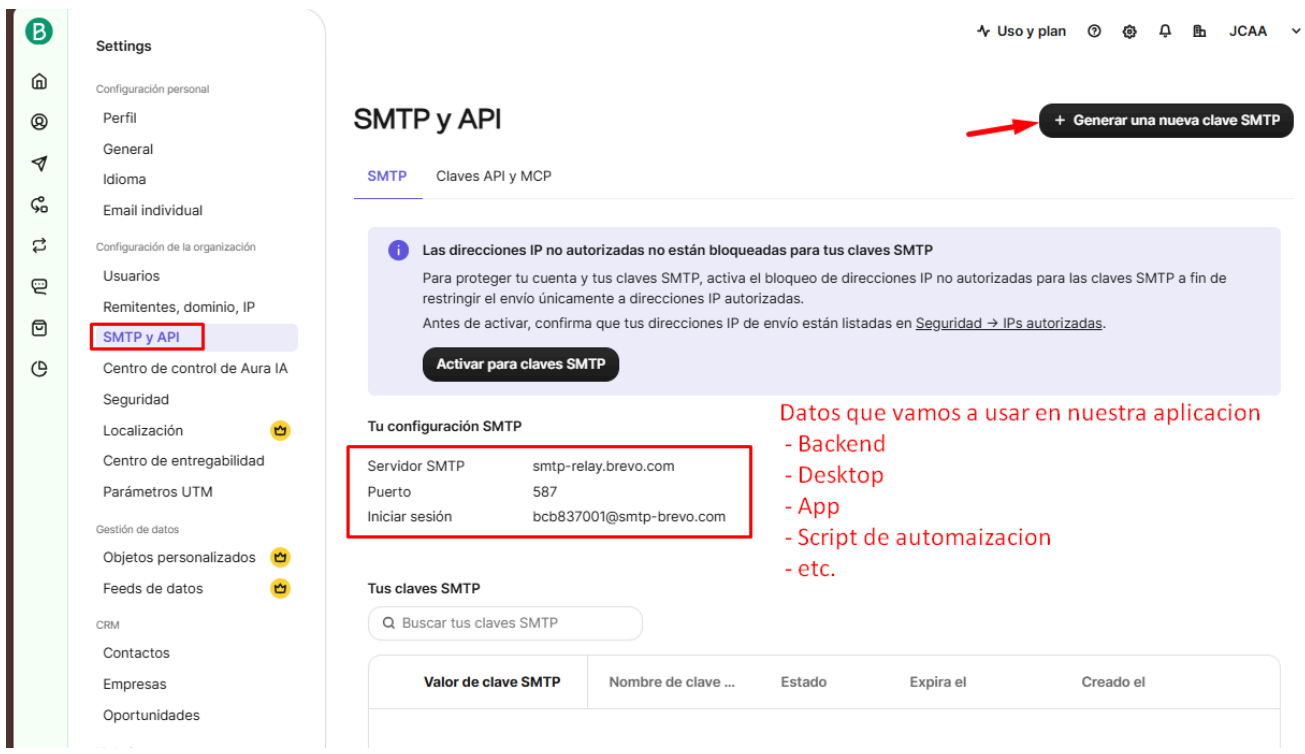


Imagen 11: Pantalla de configuración SMTP y API mostrando parámetros de servidor y botón de generación de clave.

La sección superior despliega un aviso de seguridad advirtiendo que **las direcciones IP no autorizadas no están bloqueadas para tus claves SMTP**.

Esto implica que cualquier aplicación que posea las credenciales generadas podrá autenticarse y enviar mensajes. Para proteger la cuenta contra usos no autorizados en entornos de producción, es posible restringir los envíos únicamente a direcciones IP públicas preautorizadas.

Consideraciones clave antes de activar la restricción por IP:

1. Evaluación del impacto en la seguridad: Si las claves SMTP quedan expuestas o son extraídas del código de la aplicación, un tercero podría utilizarlas desde cualquier ubicación para enviar spam a través de tu cuenta.
El bloqueo por IP previene esto limitando el origen del tráfico exclusivamente a la infraestructura autorizada.

2. Recomendaciones de uso según el entorno:

- **Entorno de desarrollo local (PC personal o red doméstica): Mantén la opción desactivada**. Dado que la dirección IP asignada por tu proveedor de internet suele ser dinámica, si activas la restricción, las solicitudes enviadas desde Spring Boot en tu equipo local serán rechazadas por Brevo.
- **Entorno de producción (Servidor en la nube como AWS, Render o VPS):** Es altamente recomendable activarla. **Antes de presionar el botón**, registra la IP elástica o fija del servidor en la sección *Seguridad → IPs autorizadas* para evitar la interrupción inmediata del servicio.

Advertencia para pruebas locales
Mientras estés desarrollando y haciendo pruebas locales desde tu equipo, mantén desactivado el bloqueo por IP. De lo contrario, se interrumpirán las pruebas de envío de correo en Spring Boot.

Paso 12: Validación telefónica para la creación de claves

Como medida de seguridad adicional y prevención de abusos, Brevo requiere verificar un número telefónico antes de permitir la emisión de credenciales de envío. Ingresa tu número telefónico y haz clic en **Enviar código**.

El sistema generará una clave secreta alfanumérica. **Cópiala e incorpórala de inmediato en el archivo de propiedades o variables de entorno de tu proyecto (como propiedad spring.mail.password en Spring Boot)**. Por motivos de seguridad, Brevo no volverá a mostrar este valor en pantalla.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

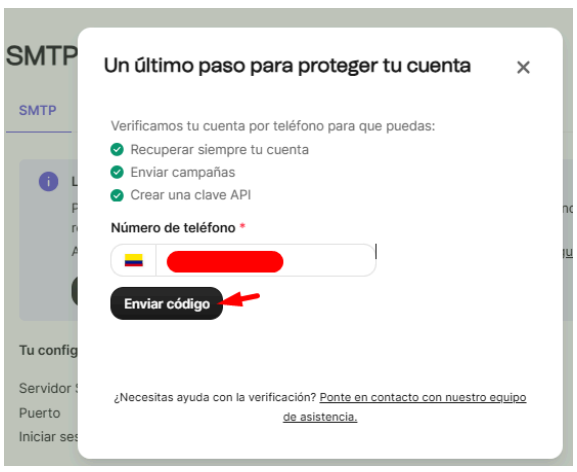


Imagen 12: Modal de verificación telefónica requerida para la creación de clave SMTP.

Paso 13: Introducción del código de verificación SMS

Introduce el código numérico recibido mediante mensaje de texto (SMS) en tu teléfono móvil y haz clic en **Verificar** para autenticar el procedimiento.



Imagen 13: Modal para ingresar el código de verificación SMS.

Paso 14: Configuración de la nueva clave SMTP

Una vez completada la verificación telefónica, se desplegará el cuadro de diálogo para la creación de la clave. Ingresa un nombre identificativo para la credencial (por ejemplo, **`mis-guias`** o **`spring-boot-app`**), selecciona la variante de longitud (Estándar) y define la vigencia de la clave antes de presionar **Generar**.



Imagen 14: Formulario para la creación de clave SMTP indicando el nombre, variante y expiración.



Imagen 15: Modal con la clave SMTP generada y advertencia de resguardo.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Paso 16: Confirmación del estado de la credencial en el panel

Al presionar **Aceptar**, serás redirigido al listado de claves SMTP, donde podrás verificar que la clave generada figura con el estado **Activa** junto con su fecha de expiración programada, lista para ser utilizada en tus desarrollos.

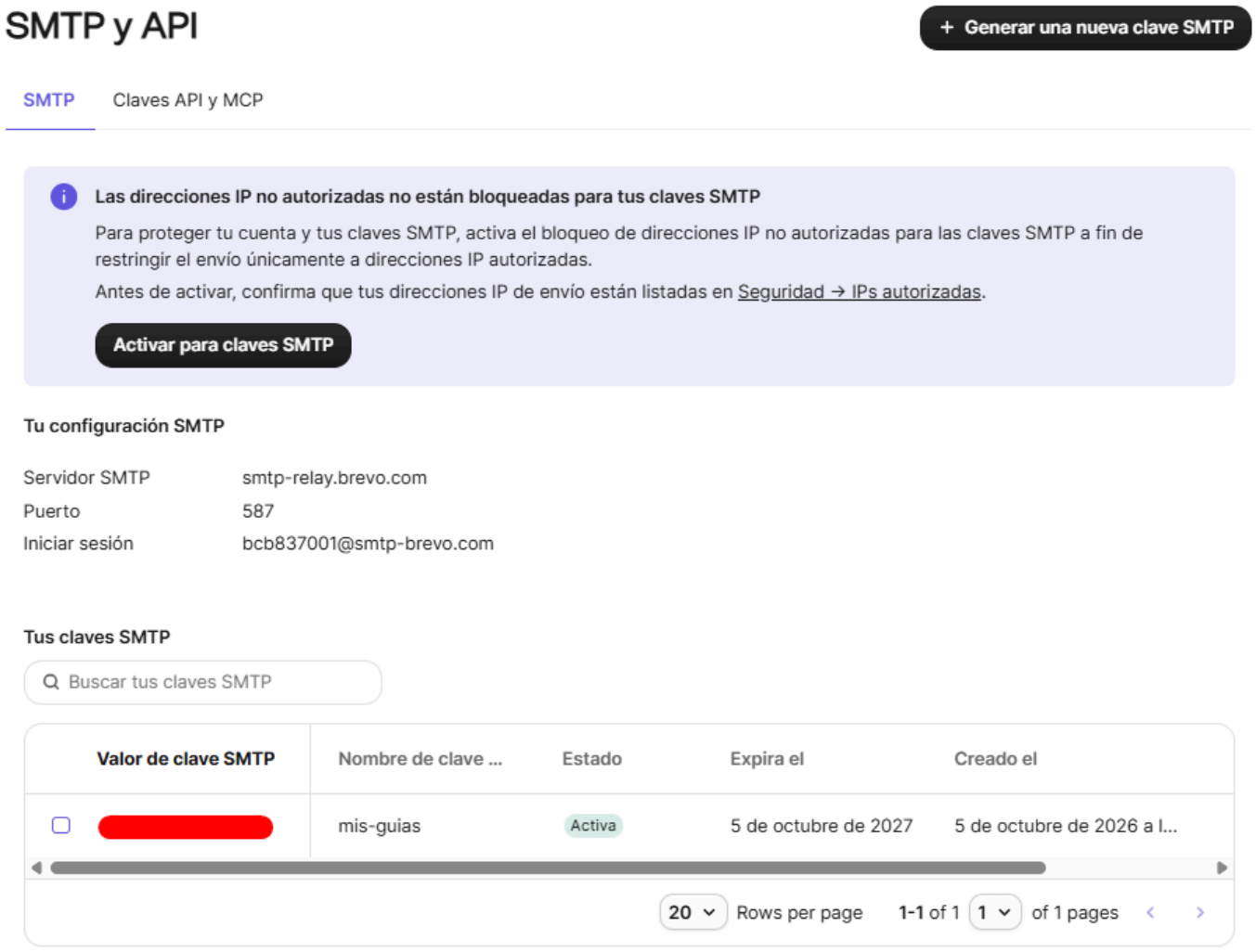


Imagen 16: Tabla de claves SMTP listando la credencial recién creada en estado activa.

Paso 17: Navegación a la pestaña de Claves API y MCP

Para aplicaciones que interactúan con Brevo mediante peticiones HTTP en lugar del protocolo SMTP tradicional, accede a la sección **SMTP y API** y haz clic en la pestaña **Claves API y MCP**. A continuación, presiona el botón **+ Generar una nueva clave API**.



Imagen 17: Pestaña de Claves API y MCP indicando el botón para generar una nueva clave API.

Paso 18: Definición de parámetros para la nueva clave API

Ingresa un nombre descriptivo para identificar el propósito de la clave API (por ejemplo, **`mis-guias-api-rest`**), selecciona la vigencia deseada y haz clic en **Generar**.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻



Imagen 18: Formulario para la creación de una clave API en Brevo.

Paso 19: Obtención y resguardo de la clave API

Se desplegará la clave API generada. Cópiala y guárdala de forma segura en las variables de entorno de tu aplicación. Esta credencial permite autenticar peticiones RESTful directas hacia los endpoints de la API de Brevo.



Imagen 19: Modal con la clave API generada en Brevo.

Paso 20: Confirmación de la clave API activa

Una vez generada, la clave aparecerá en la lista con el estado **Activa**, indicando que está lista para su integración con el backend.

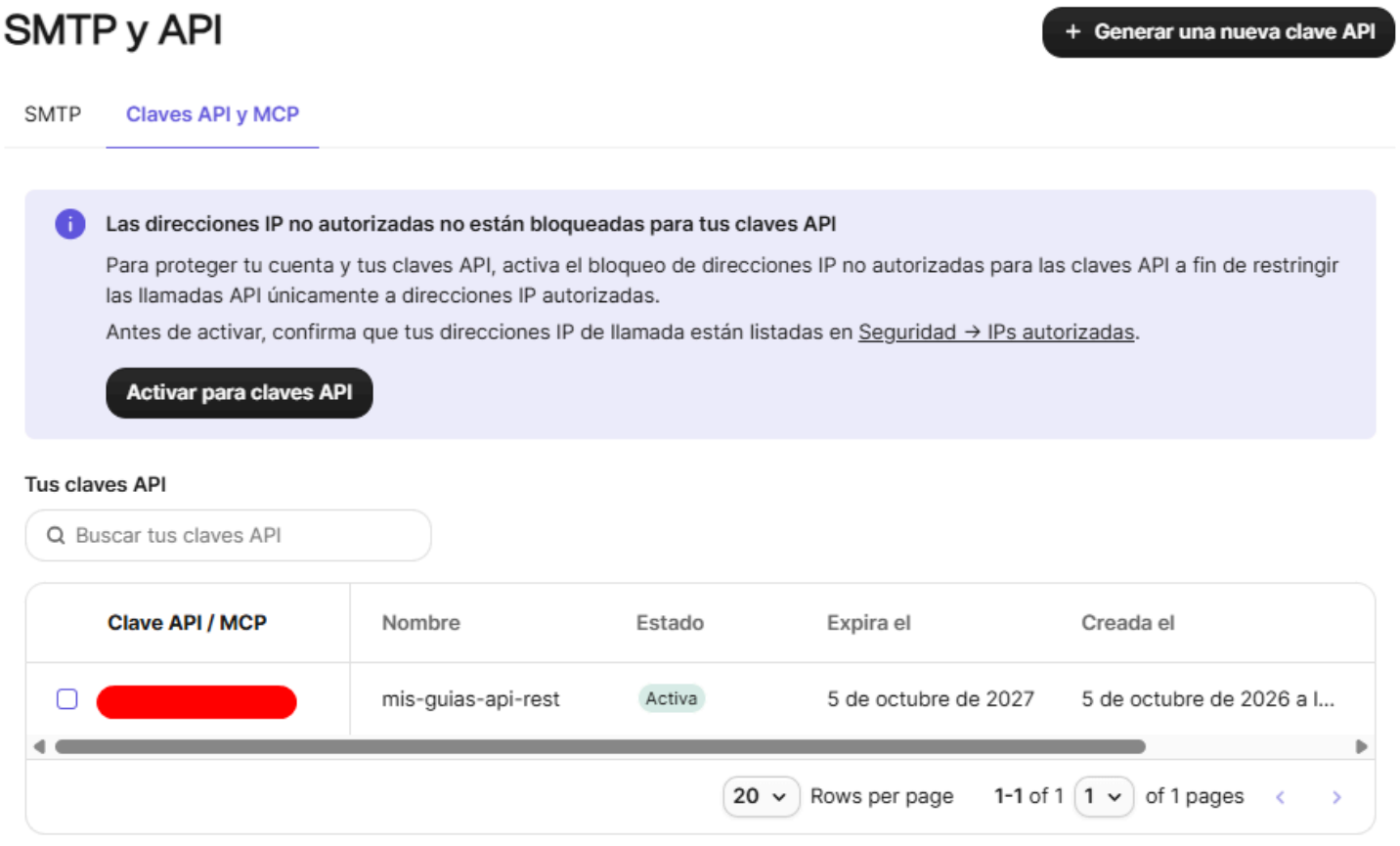


Imagen 20: Lista de claves API mostrando la credencial en estado activa.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Diferencias entre Clave SMTP y Clave API

1. Clave SMTP:

- **Objetivo:** Autenticar la conexión mediante el protocolo estándar SMTP (puertos 587, 465 o 25) para el envío de correos salientes.
- **Cuándo usarla:** Ideal para integraciones rápidas mediante clientes de correo tradicionales o librerías estándar como Spring Mail en entornos locales donde los puertos SMTP no están bloqueados.

2. Clave API (RESTful):

- **Objetivo:** Autenticar solicitudes HTTP/HTTPS directamente contra la API RESTful de Brevo.
- **Cuándo usarla:** Recomendada cuando se requiere gestión avanzada de plantillas, lectura de métricas, o cuando la aplicación está desplegada en plataformas en la nube que restringen el tráfico SMTP saliente.

¿Por qué usar API RESTful en entornos Cloud (Render, AWS, GCP)?

Muchas plataformas PaaS y IaaS en la nube, como Render, bloquean por defecto los puertos de salida SMTP estándar (25, 587, 465) para prevenir abusos y el envío no autorizado de spam desde sus servidores. Intentar conectar Spring Boot a través de SMTP en estos entornos suele resultar en errores de tiempo de espera (TimeoutException) o conexiones rechazadas.
El uso de la **API RESTful mediante HTTP (puerto 444/443 SSL)** supera completamente esta restricción, ya que el tráfico viaja como peticiones HTTPS estándar que no son bloqueadas por los firewalls de los proveedores cloud, garantizando una entrega confiable de los correos salientes.

Crear el adaptador en el proyecto para que se integre al servicio externo Brevo

NOTA IMPORTANTE:
El repositorio que aparece en las imagenes de ejemplo, actualmente ya no existe, fue renombrado con este nuevo nombre: ***users-management-microservice*** y la URL es https://github.com/arrietajohn/users-management-microservice

Si aun no haz hecho el fork del Repo, te resumiendo revisar la primera **Guia 1 - Clonar el repositorio y levantar el microservicio en local**

Flujo de trabajo en consola Git para repositorios forkeados y ramas remotas

A continuación, explicamos en detalle la secuencia de comandos ejecutados en la terminal de línea de comandos (Git Bash / CMD) para conectar tu copia local (Fork) con el repositorio original del proyecto (Upstream) y sincronizar ramas de desarrollo:

El comando **`git remote -v`** (variante con la opción *--verbose*) permite inspeccionar las conexiones remotas configuradas en tu repositorio local. Git despliega dos líneas por cada alias (por omisión, **`origin`**), una etiquetada con *(fetch)* para la obtención de datos y otra con *(push)* para el envío de cambios. Esta distinción es fundamental para verificar que el repositorio clonado apunta de manera efectiva a la dirección de tu propia cuenta forkeada antes de realizar operaciones de sincronización.

1. Inspección de los repositorios remotos configurados (git remote -v):

El comando **`git remote -v`** muestra el listado detallado de las URLs remotas vinculadas al repositorio local, especificando tanto la dirección de lectura (fetch) como la de escritura (push). Por defecto, la copia clonada desde tu cuenta personal figurará bajo el alias **`origin`**.

```
pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork
$ cd users-management-spring-boot-hexagonal-ddd/

pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git remote -v
origin  https://github.com/jarrietaa2/users-management-spring-boot-hexagonal-ddd.git (fetch)
origin  https://github.com/jarrietaa2/users-management-spring-boot-hexagonal-ddd.git (push)
```

Imagen 34: Salida del comando git remote -v mostrando las conexiones asignadas a origin.

2. Vinculación con el repositorio fuente original (git remote add upstream):

Para mantener sincronizado tu repositorio forkeado con las actualizaciones del proyecto base, es necesario agregar un segundo servidor remoto denominado habitualmente **`upstream`** mediante el comando:

```
git remote add upstream https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd.git
```

Al ejecutar nuevamente **`git remote -v`**, confirmarás que coexisten dos remotos: **`origin`** (tu copia personal donde tienes permisos de escritura) y **`upstream`** (el repositorio fuente oficial).
La instrucción **`git remote add upstream <URL>`** registra una nueva referencia remota denominada habitualmente **`upstream`**. En el modelo de desarrollo basado en forks, esta configuración es imprescindible para mantener un enlace directo hacia el repositorio

fuente original (de donde se derivó tu copia), facilitando la incorporación de nuevas características, correcciones y actualizaciones sin depender de intervenciones manuales intermedias.

```
pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git remote add upstream https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd.git

pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git remote -v
origin  https://github.com/jarrietaa2/users-management-spring-boot-hexagonal-ddd.git (fetch)
origin  https://github.com/jarrietaa2/users-management-spring-boot-hexagonal-ddd.git (push)
upstream        https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd.git (fetch)
upstream        https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd.git (push)
```

Imagen 35: Incorporación del remoto upstream enlazando el repositorio original.

3. Descarga de objetos y ramas remotas (git fetch upstream):

El comando **`git fetch upstream feature/brevo-rest-email-resilience`** consulta la infraestructura del repositorio original y descarga todas las referencias de commits, etiquetas y ramas que no existen aún en tu copia local, sin alterar directamente los archivos del área de trabajo.

Al ejecutar **`git fetch upstream <rama>`**, Git descarga la estructura de commits, objetos y referencias de ramas actualizadas presentes en el servidor remoto especificado. A diferencia de un comando como *git pull*, el proceso de *fetch* es completamente seguro e higiénico: no modifica los archivos ni mezcla el historial del área de trabajo activa, permitiendo revisar y validar las ramas remotas antes de integrarlas localmente.

```
pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git fetch upstream feature/brevo-rest-email-resilience
remote: Enumerating objects: 102, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 102 (delta 5), reused 1 (delta 1), pack-reused 85 (from 2)
Receiving objects: 100% (102/102), 32.41 KiB | 2.31 MiB/s, done.
Resolving deltas: 100% (6/6), done.
From https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd
 * branch            feature/brevo-rest-email-resilience -> FETCH_HEAD
 * [new branch]      feature/brevo-rest-email-resilience -> upstream/feature/brevo-rest-email-resilience
```

Imagen 36: Descarga de ramas y objetos del remoto mediante git fetch upstream.

```
pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git branch -r
  origin/HEAD -> origin/main
  origin/main
  upstream/feature/brevo-rest-email-resilience

pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (main)
$ git switch -c feature/brevo-rest-email-resilience --no-track upstream/feature/brevo-rest-email-resilience
Switched to a new branch 'feature/brevo-rest-email-resilience'
```

El comando **`git switch -c <nueva-rama> --no-track <remoto/rama>`** crea una nueva línea de trabajo local e commuta el contexto inmediatamente hacia ella. El indicador *--no-track* desacopla la referencia de seguimiento directa hacia el servidor `upstream`, garantizando que los commits locales de la práctica se gestionen de forma aislada e independiente en tu propio espacio personal.

Por su parte, la ejecución de **`git branch`** (y su variante remota `git branch -r`) enumera las ramas disponibles localmente o rastreadas en los servidores remotos, señalando mediante un asterisco (*) cuál es la línea de desarrollo actualmente activa en el entorno de trabajo.

```
pc@DESKTOP-TB6S72F MINGW64 ~/Documents/mis-guias/fork/users-management-spring-boot-hexagonal-ddd (feature/brevo-rest-email-resilience)
$ git branch
* feature/brevo-rest-email-resilience
  main
```

Imagen 37: Cambio y verificación de contexto de rama de trabajo en Git CLI.

Explicación: Sincronización y Configuración en IntelliJ IDEA
En la sección final del proceso de preparación del entorno de desarrollo, observamos una serie de capturas tomadas directamente desde **IntelliJ IDEA**. A continuación se analiza cada uno de estos pasos con un enfoque didáctico:

1. Verificación de la rama activa en el entorno de desarrollo

Como se ilustra en la primera imagen del IDE, en la barra superior se confirma que nos encontramos posicionados sobre la rama **`feature/brevo-rest-email-resilience`**. Esto es crucial en el flujo de trabajo de Git, ya que garantiza que cualquier cambio o prueba local se ejecute sobre la línea de trabajo que contiene el adaptador de Brevo y la resiliencia.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

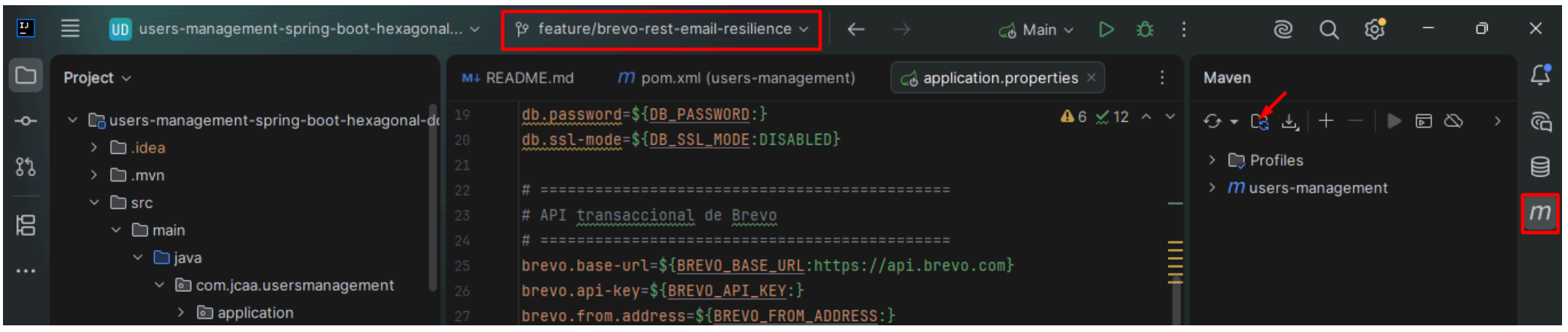


Imagen 38: Verificación de la rama activa en la barra superior de IntelliJ IDEA.

2. Generación de fuentes y sincronización del proyecto con Maven

En las capturas subsiguientes (*kix.xw53bhopfbg*, *kix.7g8qmzkzg9vz* y *kix.1h1fv3d0j8xr*), se muestra el uso de la pestaña lateral **Maven**. Al ejecutar el comando *Reload All Maven Projects* (Recargar todos los proyectos Maven), IntelliJ IDEA descarta las configuraciones en caché, descarga las nuevas dependencias (como Resilience4j) definidas en el archivo **`pom.xml`** y compila las clases generadas automáticamente. El mensaje de éxito **Generate Sources** y **Sync** valida que el proyecto Spring Boot reconozca todas las librerías necesarias sin errores de importación.

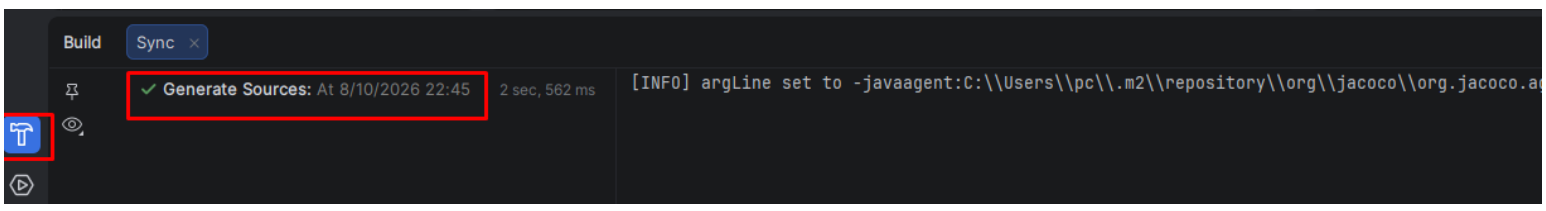


Imagen 39: Generación exitosa de fuentes compiladas desde IntelliJ IDEA.

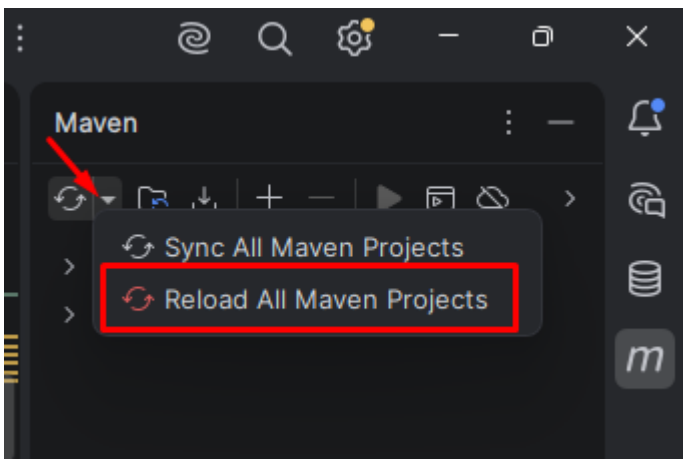


Imagen 40: Opción Reload All Maven Projects ejecutada en el panel lateral.



Imagen 41: Confirmación de sincronización del proyecto finalizada sin errores.

3. Autenticación segura mediante Personal Access Token (PAT) en Git Push

Finalmente, la captura de la terminal integrada de IntelliJ (*kix.nctooc2ayob4*) demuestra el envío de la rama local al servidor remoto mediante **`git push -u origin feature/brevo-rest-email-resilience`**. Ante la solicitud de credenciales por parte de GitHub, se ingresa el nombre de usuario y se pega el token de acceso personal (PAT) con prefijo **`ghp_`**. Esta buena práctica sustituye el uso de contraseñas convencionales y autoriza la publicación exitosa de la rama en el repositorio forkeado.

Al realizar la publicación de la rama mediante **`git push -u origin <rama>`**, GitHub exige un mecanismo de autenticación seguro. Dado que las contraseñas tradicionales de cuenta se encuentran obsoletas para operaciones Git por HTTPS, el sistema solicitará el nombre de usuario y el **Personal Access Token (PAT)** previamente creado como credencial sustitutiva. La bandera *-u* (o *--set-upstream*) enlaza de forma permanente la rama local con la rama correspondiente en tu repositorio forkeado (`origin`), facilitando futuros envíos o recepciones de cambios con comandos simplificados.

Ahora desde la terminal escribir el comando para enviar enviar la rama **feature/brevo-rest-email-resilience** del Repo local al Repo remoto:

```
git -c credential.helper= -c core.askPass= push -u origin feature/brevo-rest-email-resilience
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

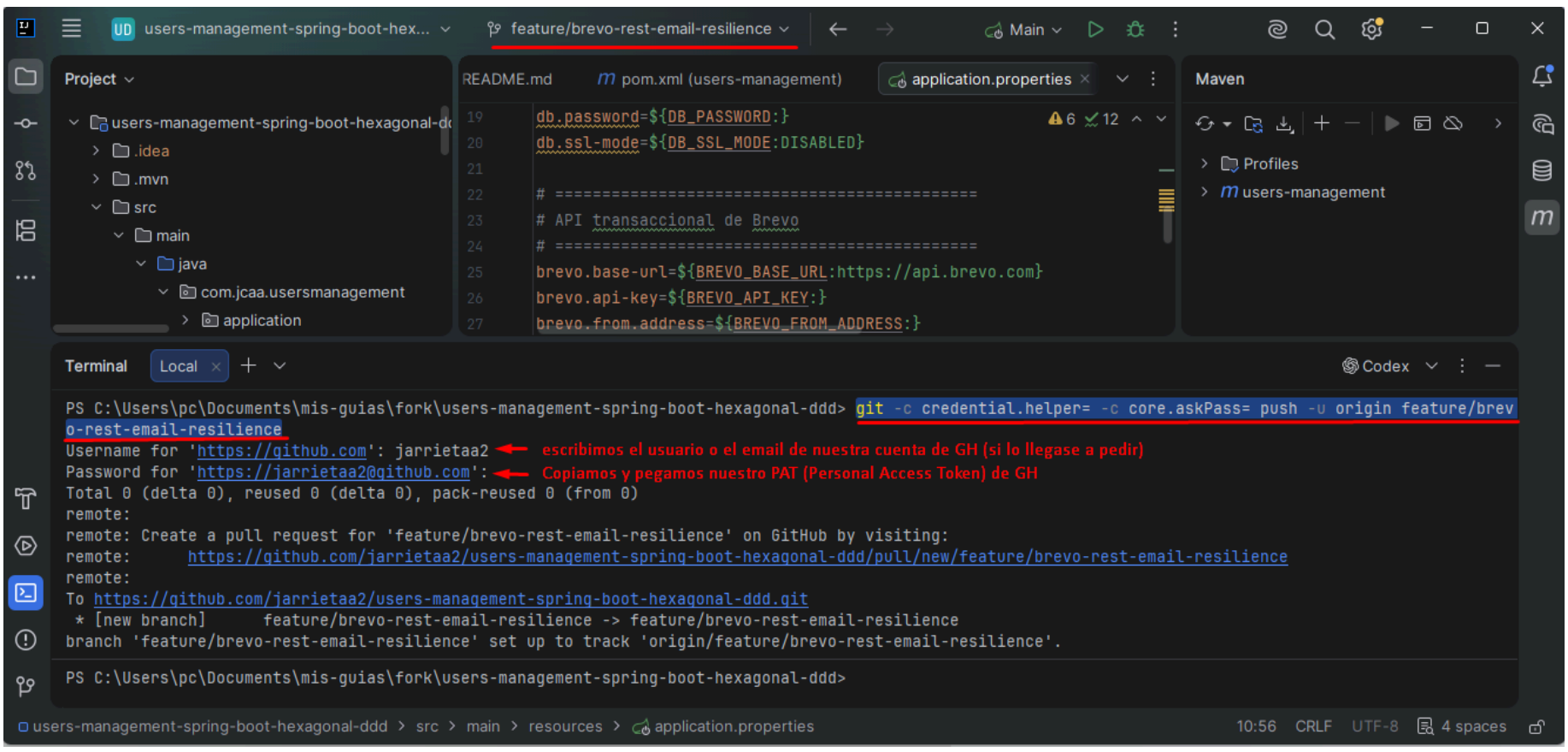


Imagen 42: Ejecución de git push ingresando el PAT en la terminal integrada.

Vamos al Repo Fork en GH, a la seccion de Pull Request, si no observamos el PR creado por GH, entonces, creamos un PR nuevo.

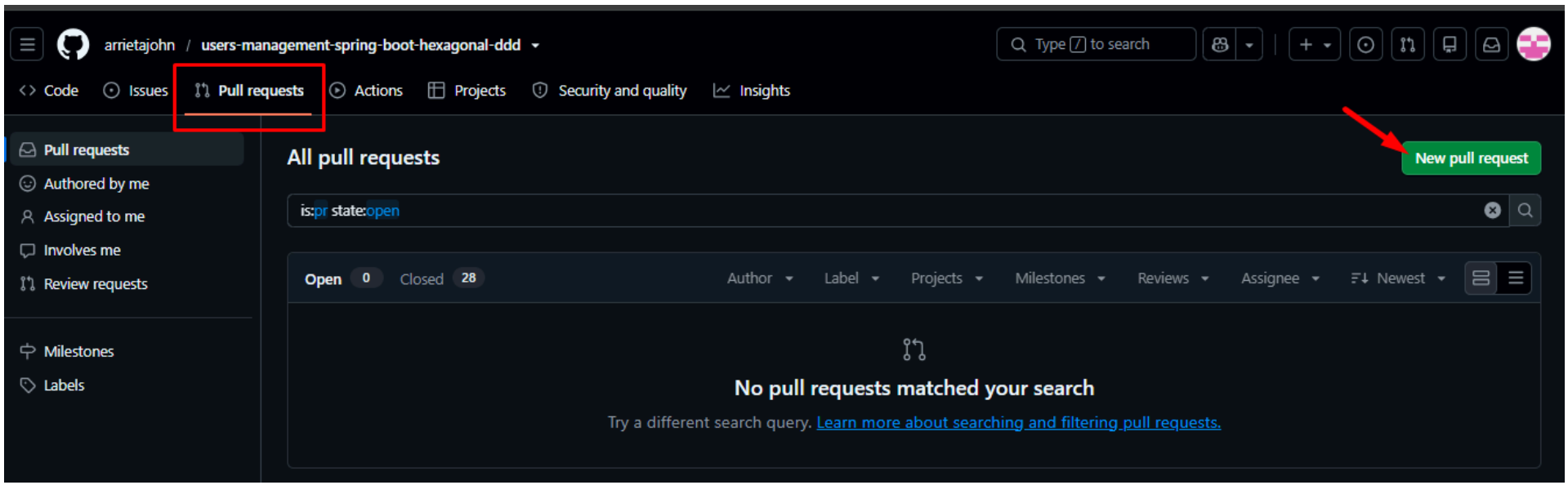


Imagen 43: Sección de Pull Requests en GitHub indicando el botón para un nuevo PR.

En **Repositorio Base** seleccionamos **nuestro Repo Fork**

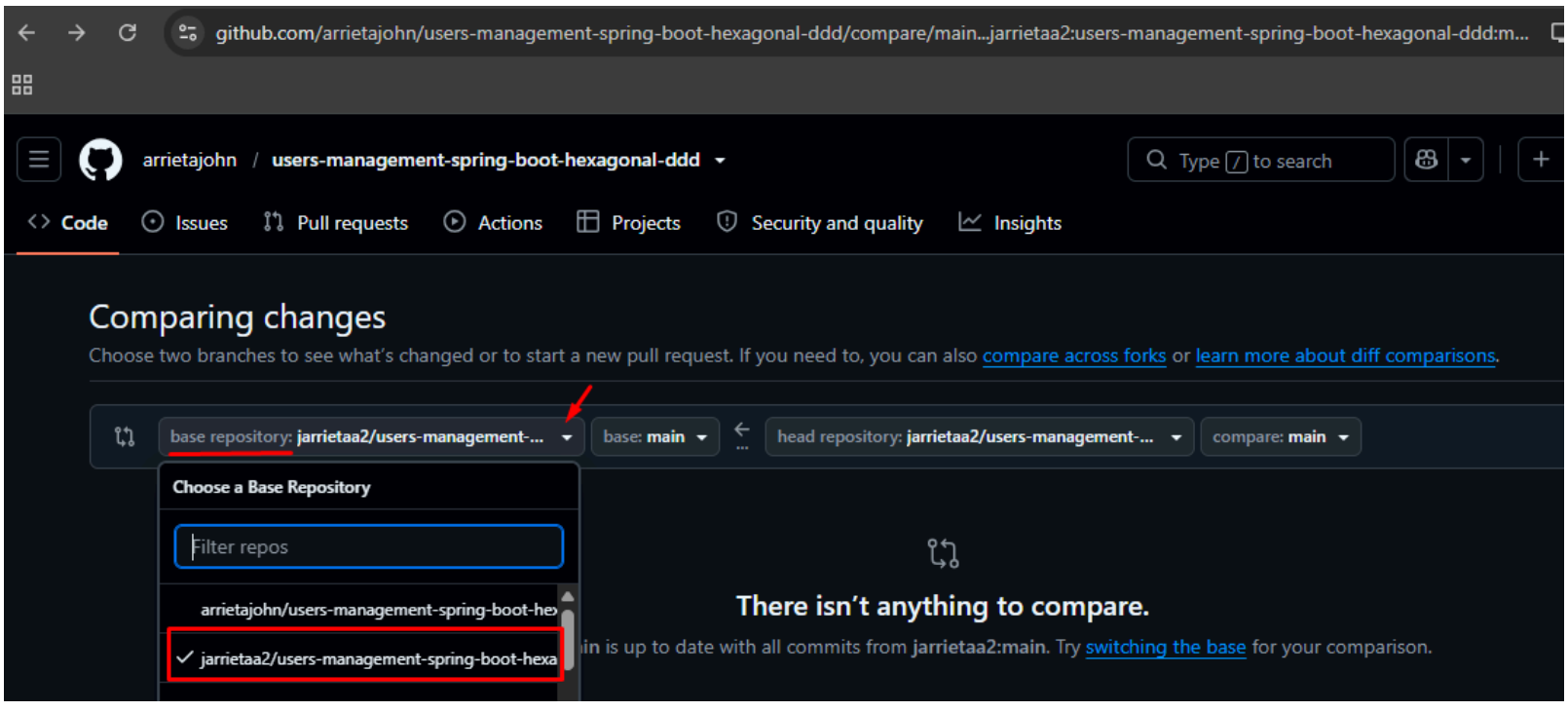


Imagen 44: Selección del repositorio base adecuado para la comparación.

Ahora en la rama a comparar seleccionamos la rama que hemos clonado: **feature/brevo-rest-email-resilience**

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

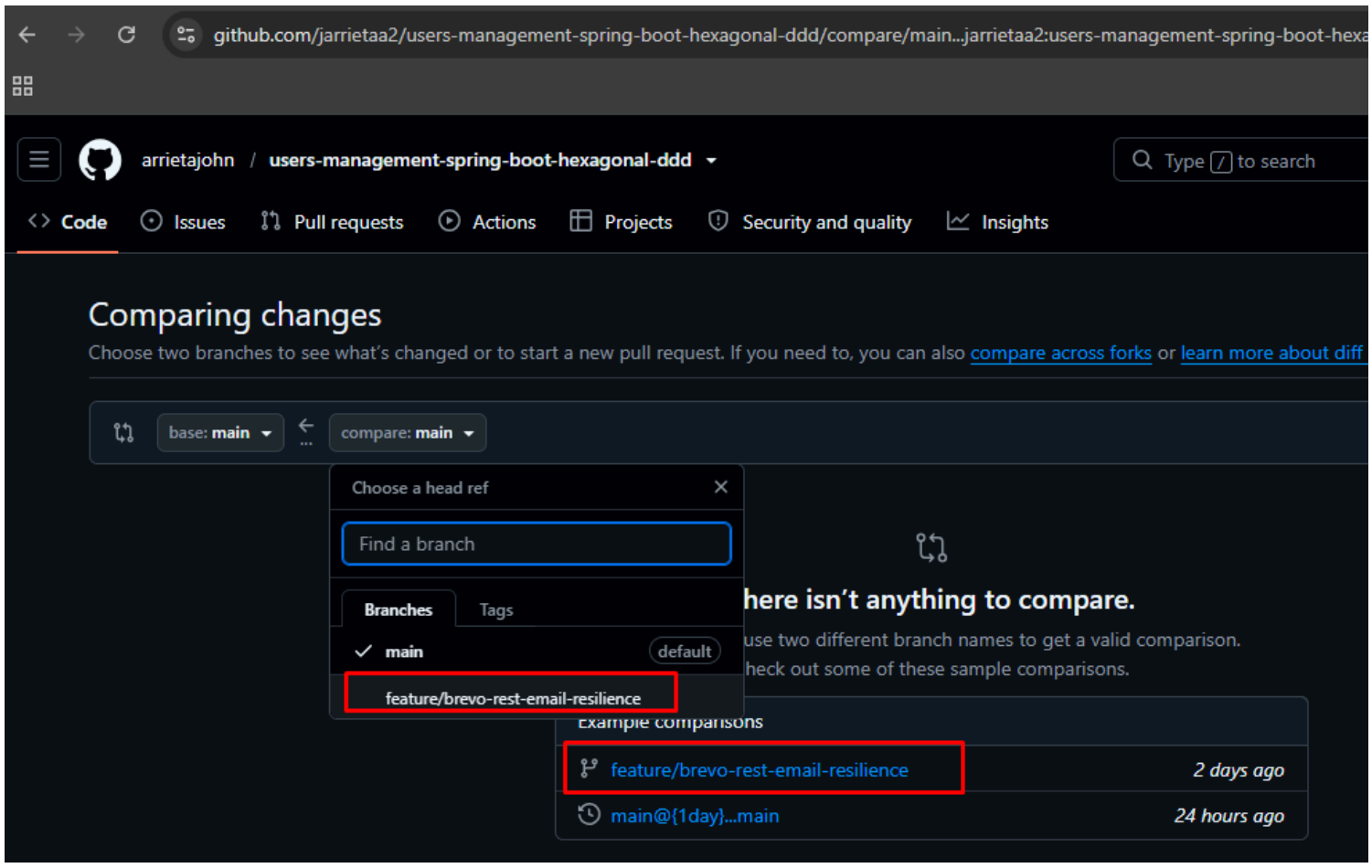


Imagen 45: Elección de la rama comparativa feature/brevo-rest-email-resilience.

Creamos el PR:

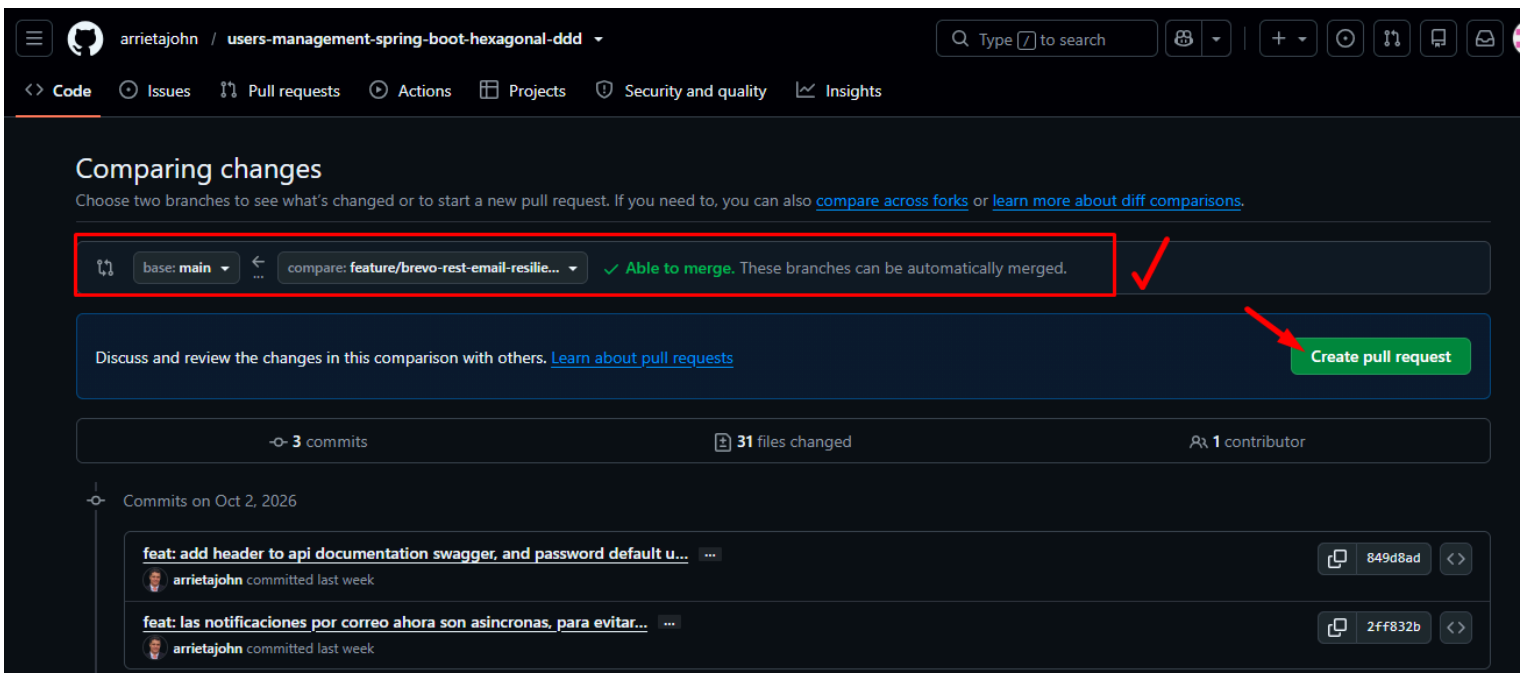


Imagen 46: Pantalla de confirmación previa a la apertura del Pull Request.

Le agregamos una descripción al PR y confírmanos que lo vamos a crear

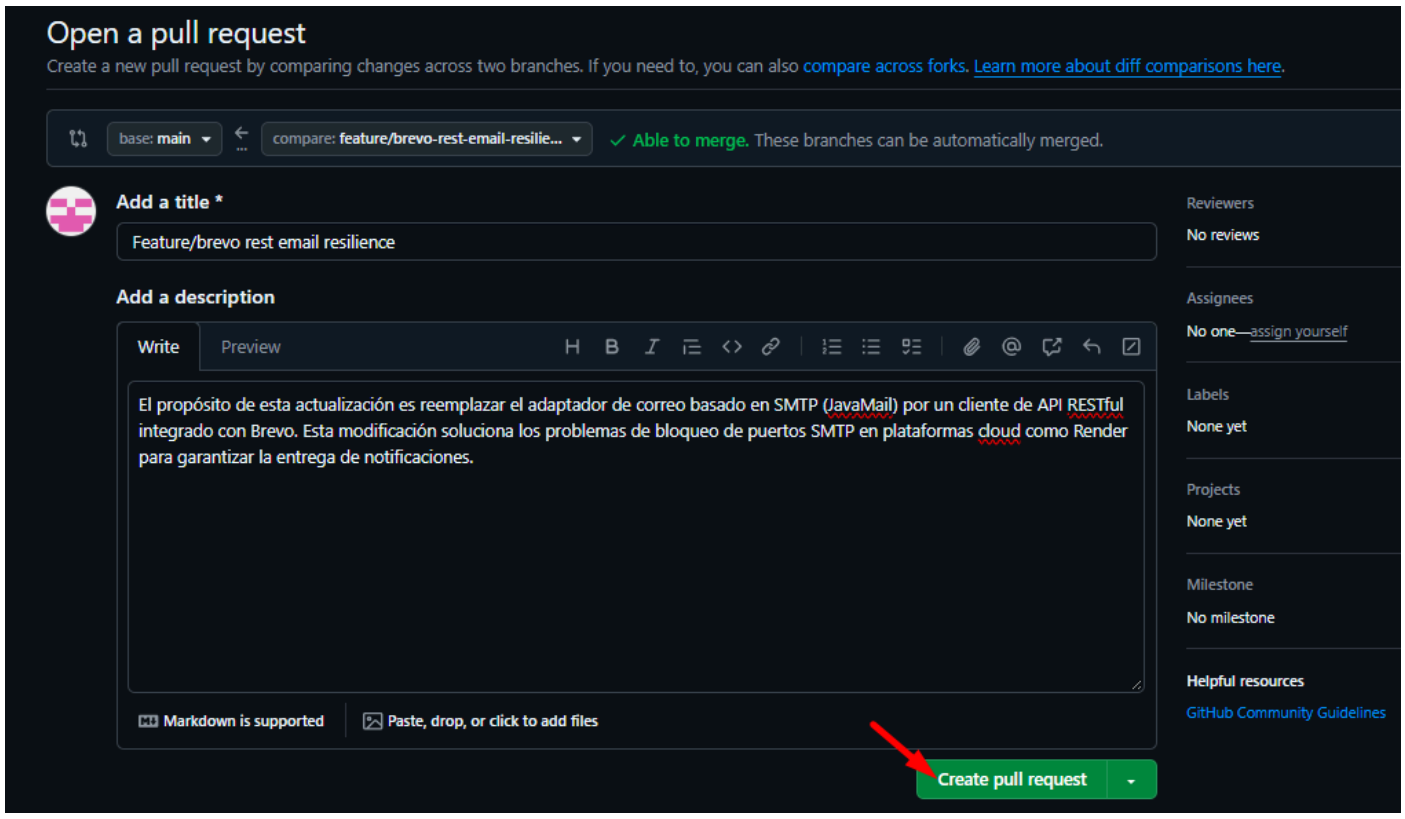


Imagen 47: Formulario de descripción y envío definitivo del Pull Request.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Pulsamos en Merge Pull Requeste para mezclar el PR en la rama main del Repo Fork

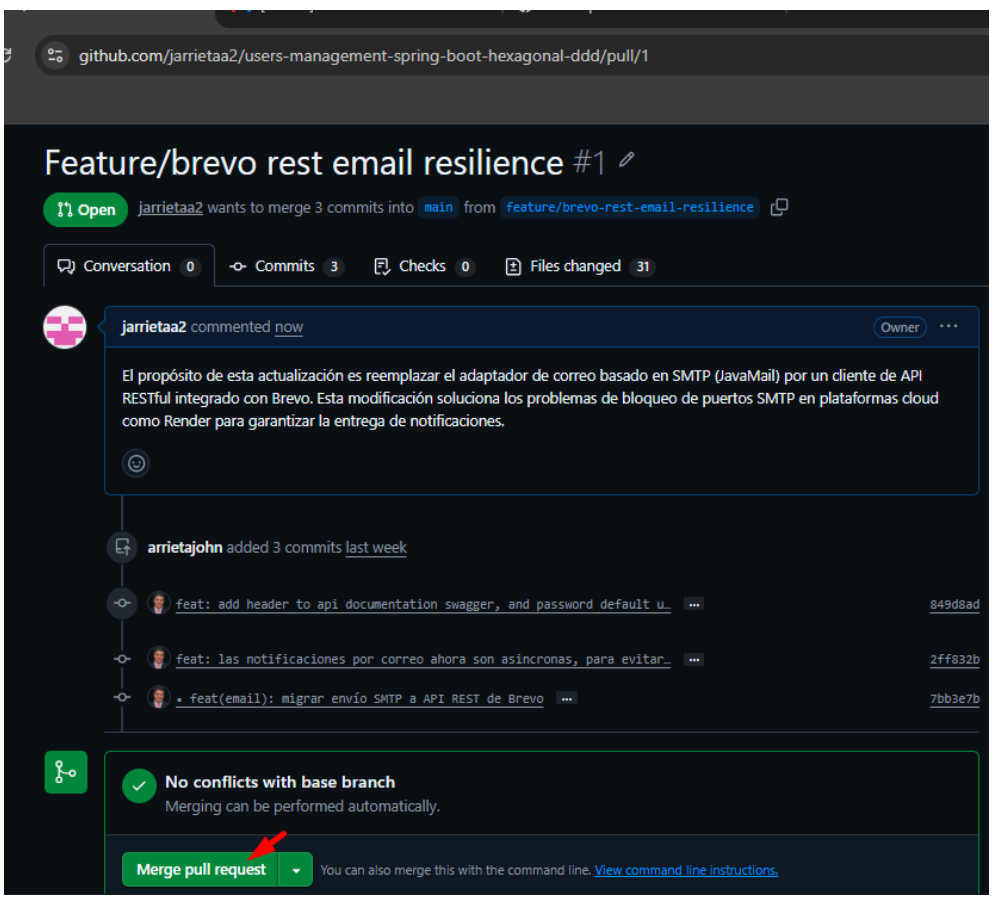


Imagen 48: Vista del Pull Request abierto habilitando la fusión automática.

Confirmamos que deseamos mezclar el PR

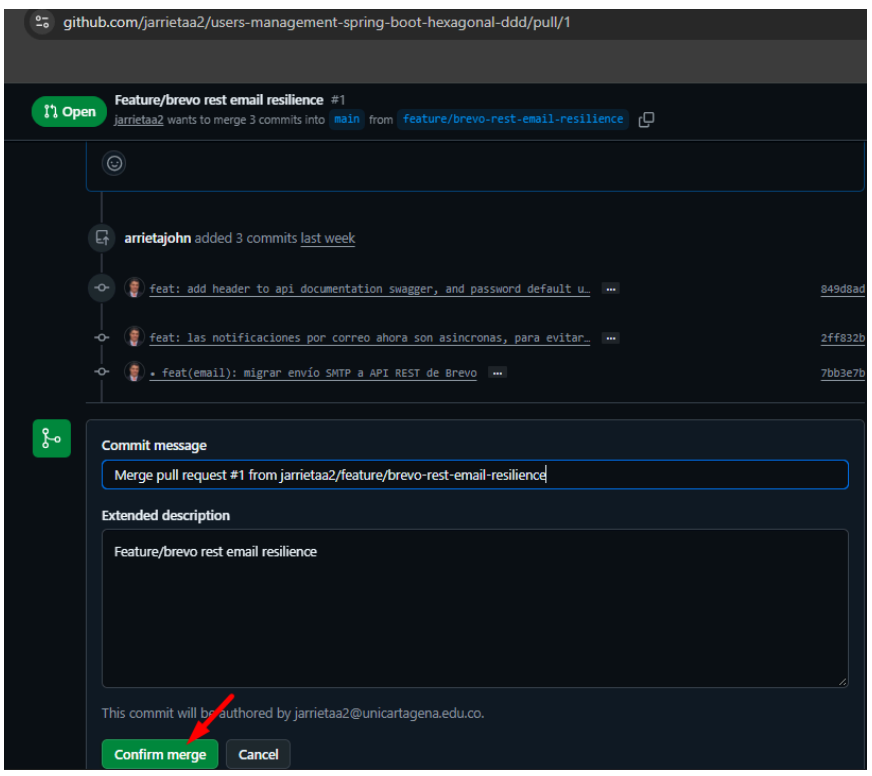


Imagen 49: Confirmación del mensaje de commit para la mezcla de ramas.

Aparece mezclado luego de unos segundos

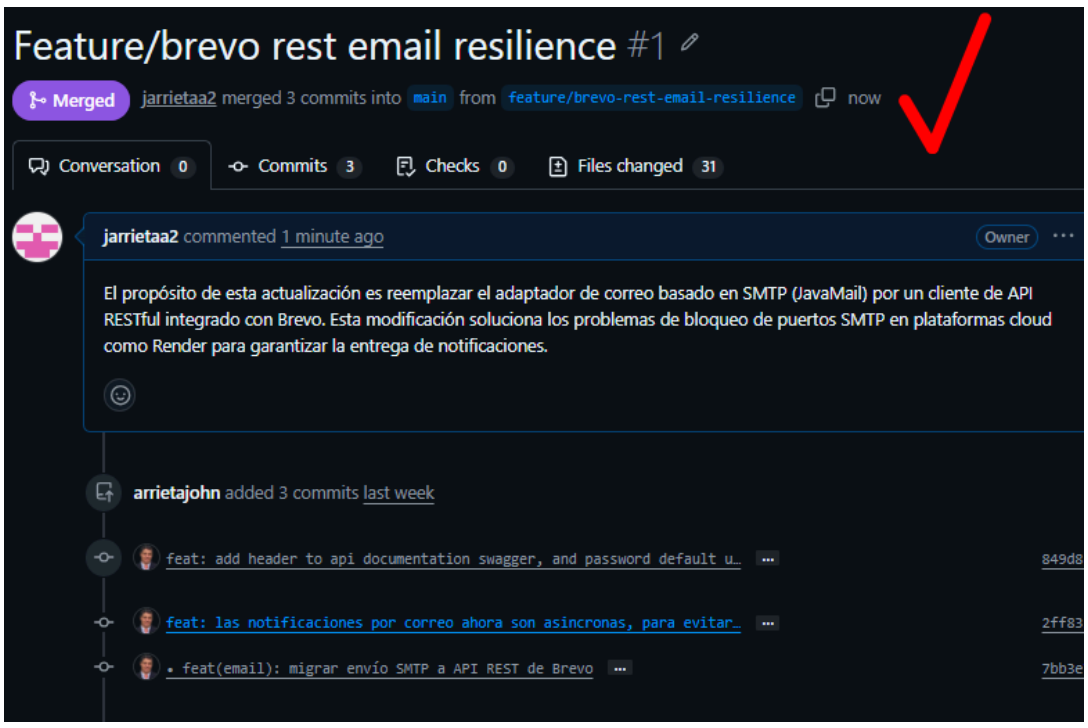


Imagen 50: Estado final mostrando la Pull Request mezclada exitosamente.

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Verificamos que ya aparezcan los cambios de la rama clonada en la rama main del Repo Fork

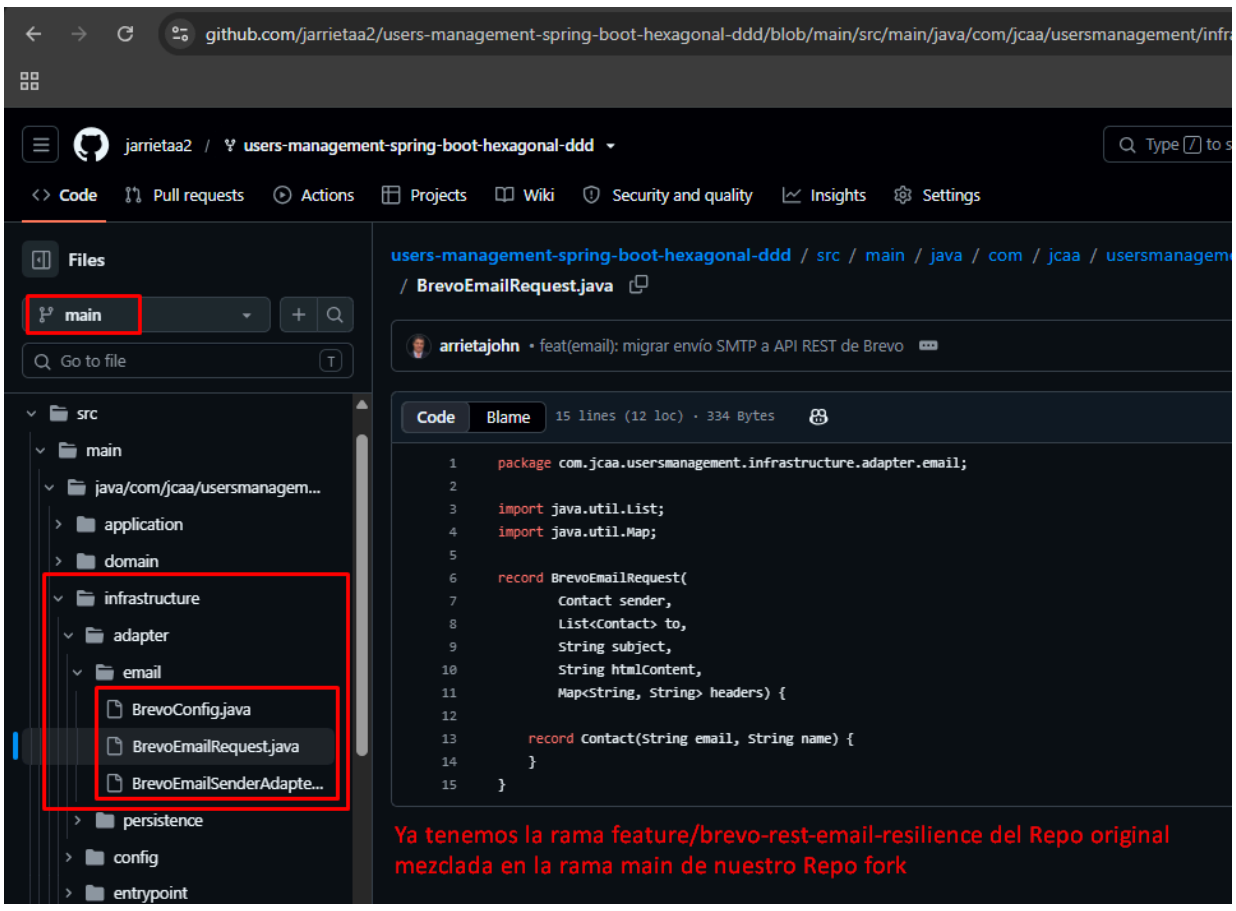


Imagen 51: Verificación del contenido de la rama main actualizando archivos.

Probar la conexión desde el local

1. Ir a Render y buscar la sección de despliegue de la aplicación.

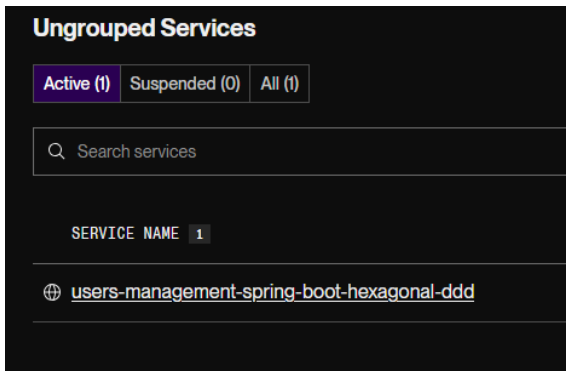


*Imagen 52: Panel de servicios de Render listando la aplicación activ**a.***

2. Seleccionar en el panel izquierdo la opción Environments

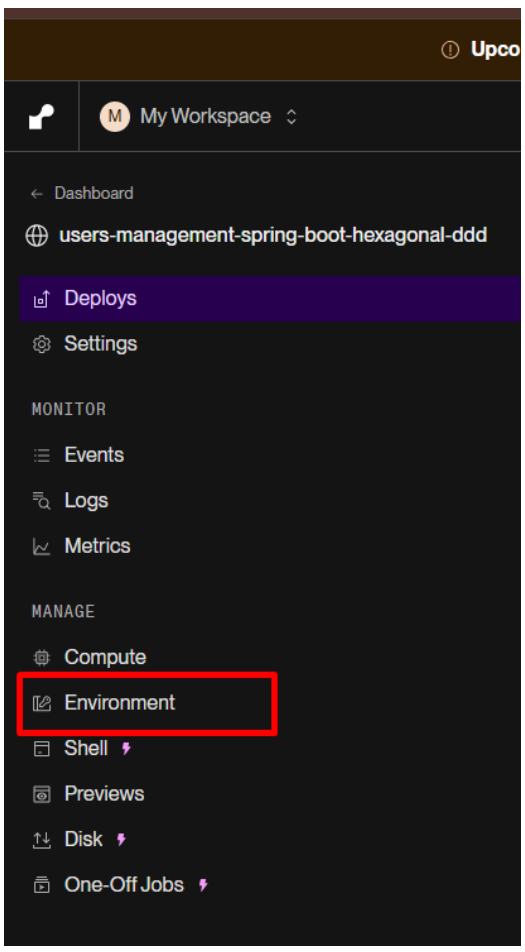


*Imagen 53: Menú lateral de Render destacando la sección Environment**.***

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

3. Busca la opción editar

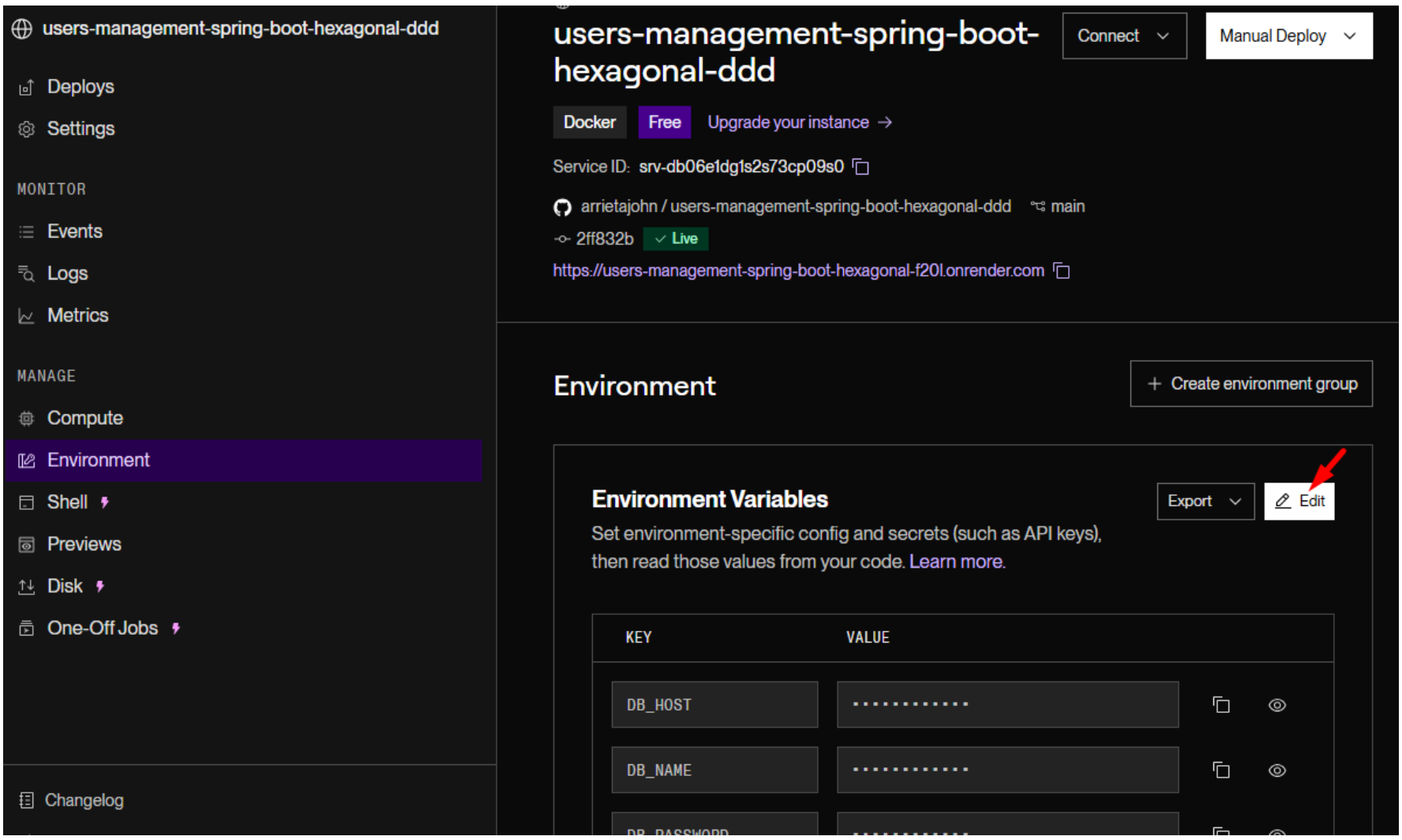


Imagen 54: Botón Edit para modificar las variables de entorno en Render.

4. Agrega las nuevas environments definidas para Brevo



Imagen 55: Adición de nombres de claves para las variables de entorno de Brevo.

5. Ve a Brevo.com , entra a la sección de remitentes, dominio, IP y copia la dirección verificada.

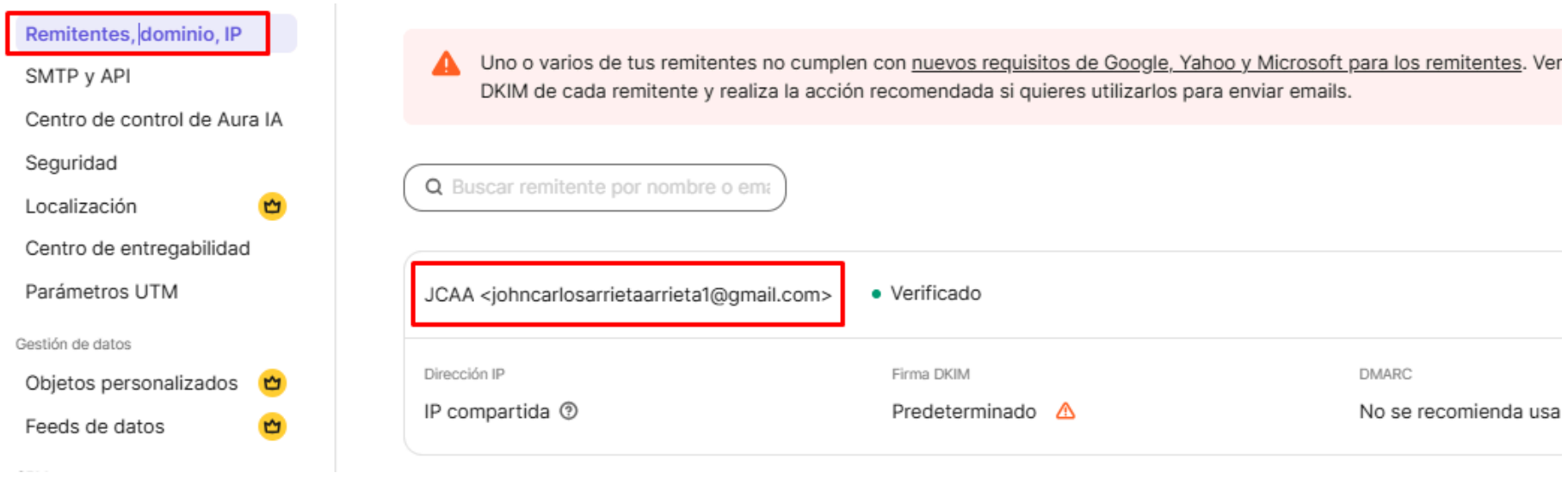


Imagen 56: Consulta y copia del remitente verificado en la cuenta de Brevo.

6. Regresa a Render / Variables de entorno y agrega los valores para cada variable de entorno y guarda los cambios

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

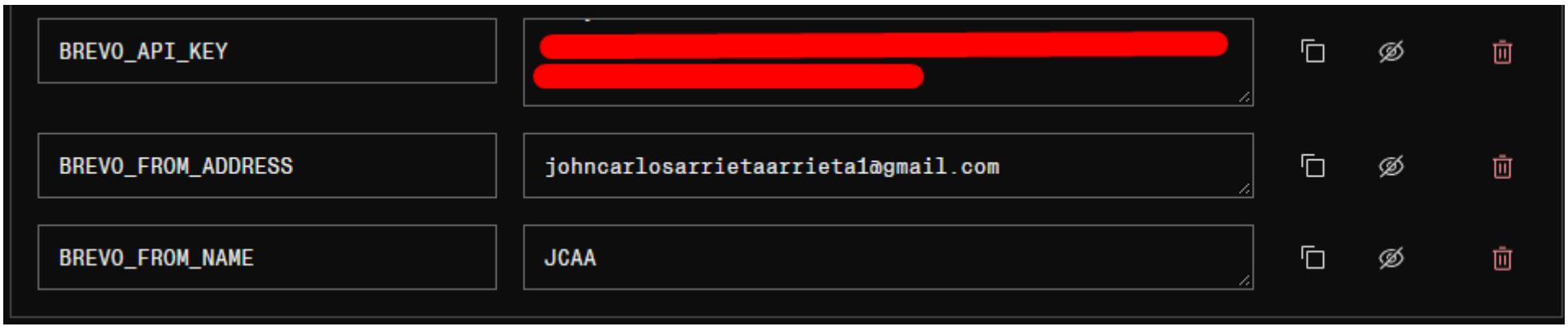


Imagen 57: Guardado de valores para las variables de entorno en Render.

7. Realiza el commit, push y merge de la rama con los cambios en local, espera que render realice el despliegue de esos cambios y prueba, pero ahora usando la URL que Render generó para la aplicación desplegada.

Configurar las variables de entorno en Render y probar

Construir el puerto y adaptador para el API de Brevo.

1. Crear un archivo de configuración para cargar las variables de entorno

A continuación, se define el archivo de plantilla para las variables de entorno utilizadas por Docker Compose. Su propósito es servir como guía para que los desarrolladores conozcan qué variables configurables se necesitan (como la clave de API de Brevo, remitentes y credenciales de base de datos) sin exponer los datos sensibles reales en el control de versiones.

compose.env.example

```
# Copie este archivo como .env antes de ejecutar Docker Compose.
# Nunca agregue el archivo .env real al repositorio.

DB_PASSWORD=REEMPLAZAR
MYSQL_ROOT_PASSWORD=REEMPLAZAR
JWT_SECRET=REEMPLAZAR_CON_BASE64_DE_32_BYTES
BREVO_API_KEY=xkeysib-REEMPLAZAR
BREVO_FROM_ADDRESS=remitente-verificado@example.com
BREVO_FROM_NAME=Gestion de Usuarios
JWT_EXPIRATION_SECONDS=900
```

Este archivo sirve como plantilla de configuración para las propiedades locales de Spring Boot. Su objetivo es documentar las claves requeridas para la conexión a la base de datos MySQL, el secreto JWT y los parámetros de Brevo, permitiendo que cada desarrollador cree su propia versión local cifrada o privada.

application-secrets.properties.example

```
# Copie este archivo como .environments/application-secrets.properties.
# Reemplace todos los valores de ejemplo. Nunca versionar el archivo real.

# Base de datos local
db.host=localhost
db.port=3306
db.name=crud_usuarios
db.username=root
db.password=REEMPLAZAR

# JWT: Base64 de al menos 32 bytes aleatorios
jwt.secret=REEMPLAZAR

# Brevo
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
brevo.local.api-key=xkeysib-REEMPLAZAR
brevo.local.from.address=remitente-verificado@server.com
brevo.local.from.name=Gestion de Usuarios
```

2. Crear archivo locales (no se deben subir al repositorio remoto) que tendrán las credenciales y parametros de conexion reales

El archivo de propiedades secrets locales almacena las credenciales reales de desarrollo. Su propósito es proveer a la aplicación Spring Boot los valores concretos para autenticación en la base de datos local y la clave de API privada de Brevo durante la ejecución en entorno de desarrollo local.

.environments/application-secrets.properties



Imagen 58: Archivo de propiedades de secretos locales configurado en la práctica.

El archivo .env define las variables de entorno locales que alimentan a los servicios orquestados por Docker Compose, garantizando que el contenedor de la aplicación y la base de datos dispongan de los secretos de conexión necesarios sin incluirlos directamente en los archivos de configuración versionados.

.env

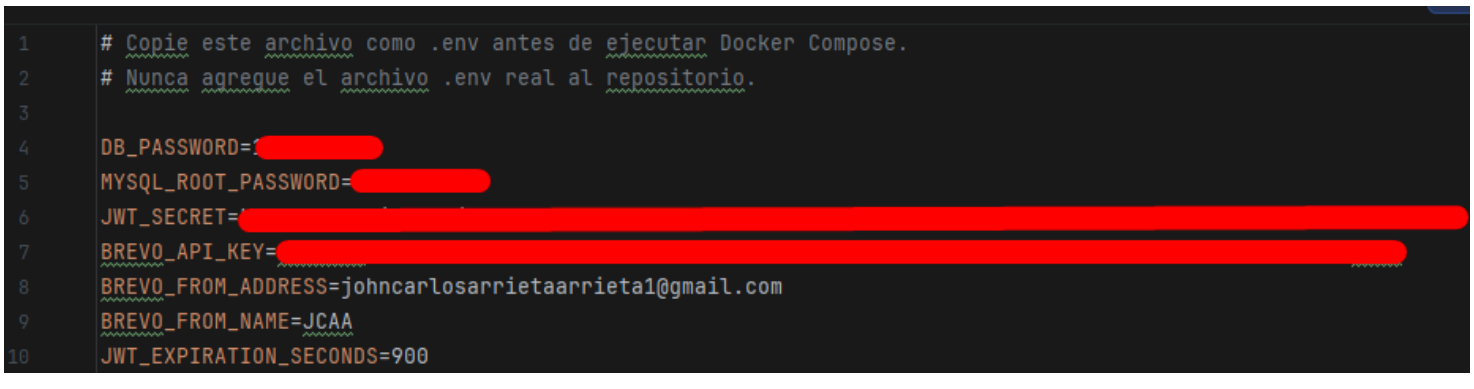


Imagen 59: Configuración de variables en el archivo .env para Docker Compose.

3. Agregar los archivos de variables de entorno con parámetros y secretos (token, APIKEYS, passwords, etc) en .gitignore, .git\info\exclude y .dockerignore

A continuación, se detalla la adición de los patrones de archivos privados al archivo .dockerignore. La lógica de esta configuración es prevenir que los archivos con claves privadas o secretos locales sean empaquetados dentro de la imagen Docker generada, garantizando la seguridad en la distribución del contenedor.

Al final de **.dockerignore**

```
# Secretos locales. Nunca deben versionarse.
/.environments/
/.env
```

La siguiente adición al archivo .gitignore evita que las credenciales locales y archivos de configuración privados sean rastreados o subidos al repositorio de Git, protegiendo las claves API de posibles filtraciones públicas.

Al final de **.gitignore**

```
.environments
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
.env
```

Esta regla en la exclusión local de Git asegura de forma complementaria que los archivos de entorno locales no se incluyan accidentalmente en commits locales, actuando como una capa defensiva adicional en la gestión de secretos.

Al final de **.git\info\exclude**

```
/.environments/
/.environments.example/
```

4. Definir las variables de entorno en el archivo de carga dinámica de environments de Render.

El archivo render.yaml especifica la infraestructura como código para Render. La siguiente adición declara la presencia de las variables de entorno de Brevo en el entorno de despliegue, indicando que su sincronización automática está desactivada para requerir que los valores reales se configuren manualmente de forma segura en el panel cloud.

Al final de **render.ymal**

```
- key: BREVO_API_KEY
 sync: false
- key: BREVO_FROM_ADDRESS
 sync: false
- key: BREVO_FROM_NAME
```

5. Agregar las configuraciones necesarias de variables de entrono para Docker compose:

El archivo compose.yaml orquesta los contenedores Docker para la aplicación y la base de datos MySQL. En esta sección se configuran las variables de entorno del servicio API para inyectar la clave API de Brevo, los parámetros de remitente, el secreto JWT y las reglas de dependencia de salud (healthcheck) de la base de datos antes de iniciar la aplicación.

compose.ymal

```
services:
 mysql:
   image: mysql:8.4
   environment:
     MYSQL_DATABASE: crud_usuarios
     MYSQL_USER: users_app
     MYSQL_PASSWORD: ${DB_PASSWORD:?Defina DB_PASSWORD en el archivo .env local}
     MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Defina MYSQL_ROOT_PASSWORD en el archivo .env local}
   ports:
     - "127.0.0.1:3306:3306"
   volumes:
     - mysql-data:/var/lib/mysql
     - ./src/main/resources/schema.sql:/docker-entrypoint-initdb.d/001-schema.sql:ro
   healthcheck:
     test:
       - CMD-SHELL
       - mysqladmin ping --host=127.0.0.1 --user=root --password=$${MYSQL_ROOT_PASSWORD}
     interval: 5s
     timeout: 5s
     retries: 20
     start_period: 20s
   restart: unless-stopped

api:
   build:
     context: .
     dockerfile: Dockerfile
   image: users-management-api:local
   environment:
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
    PORT: "8080"
    DB_HOST: mysql
    DB_PORT: "3306"
    DB_NAME: crud_usuarios
    DB_USERNAME: users_app
    DB_PASSWORD: ${DB_PASSWORD:?Defina DB_PASSWORD en el archivo .env local}
    DB_SSL_MODE: DISABLED
    BREVO_API_KEY: ${BREVO_API_KEY:?Defina BREVO_API_KEY en el archivo .env local}
    BREVO_FROM_ADDRESS: ${BREVO_FROM_ADDRESS:?Defina BREVO_FROM_ADDRESS en el archivo .env local}
    BREVO_FROM_NAME: ${BREVO_FROM_NAME:-Gestion de Usuarios}
    JWT_SECRET: ${JWT_SECRET:?Defina JWT_SECRET en el archivo .env local}
    JWT_EXPIRATION_SECONDS: ${JWT_EXPIRATION_SECONDS:-900}
  ports:
    - "127.0.0.1:8080:8080"
  depends_on:
    mysql:
      condition: service_healthy
  restart: unless-stopped
  security_opt:
    - no-new-privileges:true

volumes:
 mysql-data:
```

6. Enlazar el archivo de configuración con el archivo de propiedades de spring boot

El archivo application.properties es el centro de configuración de Spring Boot. Su lógica principal en este paso es enlazar el archivo externo de propiedades secrets, así como definir los valores por defecto y variables de entorno para la conexión a MySQL, el secreto JWT y las políticas de reintento, tiempos de espera y resiliencia (Circuit Breaker) para el cliente de Brevo.

src/main/resources/application.properties

```
# ==============================================
# Spring Boot
# ==============================================
spring.application.name=users-management
server.port=${PORT:8080}

# ENLAZAR EL ARCHIVO PRIVADO CON VARIABLES DE ENTORNO Y SECRETOS REALES
spring.config.import=optional:file:./.environments/application-secrets.properties

# DataSource se configura manualmente con @Configuration
spring.autoconfigure.exclude[0]=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
spring.autoconfigure.exclude[1]=org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAut
oConfiguration

# ==============================================
# Configuracion de Base de Datos MySQL
# ==============================================
db.host=${DB_HOST:localhost}
db.port=${DB_PORT:3306}
db.name=${DB_NAME:crud_usuarios}
db.username=${DB_USERNAME:root}
db.password=${DB_PASSWORD:}
db.ssl-mode=${DB_SSL_MODE:DISABLED}

# ==============================================
# API transaccional de Brevo
# ==============================================
brevo.base-url=${BREVO_BASE_URL:https://api.brevo.com}
brevo.api-key=${BREVO_API_KEY:}
brevo.from.address=${BREVO_FROM_ADDRESS:}
brevo.from.name=${BREVO_FROM_NAME:Gestion de Usuarios}
brevo.timeout.connect=${BREVO_CONNECT_TIMEOUT:2s}
brevo.timeout.read=${BREVO_READ_TIMEOUT:5s}
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ©+ⓘ+↻

```
brevo.retry.max-attempts=${BREVO_RETRY_MAX_ATTEMPTS:3}
brevo.retry.initial-delay=${BREVO_RETRY_INITIAL_DELAY:500ms}
brevo.retry.multiplier=${BREVO_RETRY_MULTIPLIER:2.0}
brevo.circuit-breaker.failure-rate-threshold=${BREVO_CIRCUIT_FAILURE_THRESHOLD:50}
brevo.circuit-breaker.sliding-window-size=${BREVO_CIRCUIT_WINDOW_SIZE:10}
brevo.circuit-breaker.minimum-calls=${BREVO_CIRCUIT_MINIMUM_CALLS:5}
brevo.circuit-breaker.open-duration=${BREVO_CIRCUIT_OPEN_DURATION:30s}

# ================================================
# Seguridad JWT
# En produccion defina JWT_SECRET con al menos 32 bytes aleatorios codificados en Base64.
# ================================================
jwt.secret=${JWT_SECRET:}
jwt.expiration-seconds=${JWT_EXPIRATION_SECONDS:900}
```

7. Crear las plantillas HTML del cuerpo de las notificaciones por correo.

La siguiente plantilla HTML define la estructura del correo electrónico enviado automáticamente cuando se crea un nuevo usuario. Contiene los marcadores de posición (placeholders) que serán reemplazados dinámicamente con el nombre, correo, rol y contraseña temporal generada para el usuario.

src/main/resources/templates/**user-created.html**

```
<html>
<body>
<h2>Hola, {{name}}</h2>
<p>Tu cuenta ha sido creada exitosamente.</p>
<ul>
 <li><strong>Email:</strong> {{email}}</li>
 <li><strong>Contraseña temporal:</strong> {{password}}</li>
 <li><strong>Rol:</strong> {{role}}</li>
</ul>
<p>Por favor cambia tu contraseña al ingresar.</p>
</body>
</html>
```

Esta plantilla HTML define el formato del correo de notificación cuando la información de un usuario es actualizada en el sistema. Utiliza etiquetas dinámicas para mostrar el estado actualizado de la cuenta, garantizando que el usuario reciba la confirmación de los cambios realizados.

src/main/resources/templates/**user-updated.html**

```
<html>
<body>
<h2>Hola, {{name}}</h2>
<p>Tu cuenta ha sido actualizada exitosamente.</p>
<ul>
 <li><strong>Email:</strong> {{email}}</li>
 <li><strong>Rol:</strong> {{role}}</li>
 <li><strong>Estado:</strong> {{status}}</li>
</ul>
</body>
</html>
```

8. Agregar al POM.xml las librerías (dependencias) necesarias (para resiliencia)

El archivo pom.xml administra las dependencias del proyecto Maven. En esta sección se agregan los módulos de Resilience4j (retry y circuitbreaker) necesarios para dotar de tolerancia a fallos y resiliencia a las llamadas HTTP realizadas hacia la API RESTful de Brevo.

POM.xml

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

Dentro del elemento **<properties>** agregamos una variable xml para la versión de la librería que usamos para manejar la resiliencia

```
<properties>

<resilience4j.version>2.2.0</resilience4j.version>

</properties>
```

Dentro del elemento **<dependencies>** agregamos las dependencias para descargar las librerías **Resilience4j** para trabajar con **Retry** y **Circuitbraker**

```
<dependencies>

   <!-- Resiliencia para el cliente HTTP de Brevo -->
   <dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-retry</artifactId>
    <version>${resilience4j.version}</version>
   </dependency>
   <dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-circuitbreaker</artifactId>
    <version>${resilience4j.version}</version>
   </dependency>

</dependencies>
```

9. Crear las clases Excepciones de dominio necesarias

La clase abstracta DomainException sirve como la clase base para todas las excepciones de negocio dentro del núcleo del dominio. Su objetivo es unificar el manejo de errores de lógica de negocio desacoplándolos de las capas de infraestructura o frameworks externos.

src/main/java/com/jcaa/usersmanagement/domain/exception/**DomainException.java**

```
package com.jcaa.usersmanagement.domain.exception;

public abstract class DomainException extends RuntimeException {

 protected DomainException(final String message) {
   super(message);
 }

 protected DomainException(final String message, final Throwable cause) {
   super(message, cause);
 }
}
```

src/main/java/com/jcaa/usersmanagement/domain/exception/**InvalidUserEmailException**.java

```
package com.jcaa.usersmanagement.domain.exception;

public final class InvalidUserEmailException extends DomainException {

 private static final String MESSAGE_EMPTY = "The user email must not be empty.";
 private static final String MESSAGE_INVALID_FORMAT = "The user email format is invalid: '%s'.";

 private InvalidUserEmailException(final String message) {
   super(message);
 }

 public static InvalidUserEmailException becauseValueIsEmpty() {
   return new InvalidUserEmailException(MESSAGE_EMPTY);
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
 }

 public static InvalidUserEmailException becauseFormatIsInvalid(final String email) {
   return new InvalidUserEmailException(String.format(MESSAGE_INVALID_FORMAT, email));
 }
}
```

Esta excepción de dominio es lanzada cuando el correo electrónico proporcionado no cumple con las reglas de negocio (por estar vacío o presentar un formato no válido). Incluye métodos de fábrica estáticos para expresar claramente la causa del fallo.

src/main/java/com/jcaa/usersmanagement/domain/exception/**EmailSenderException**.java
La clase EmailSenderException representa una excepción del dominio que se dispara cuando ocurre un error insuperable durante la transmisión de correos o en la comunicación con la API de Brevo, permitiendo capturar el error y manejarlo adecuadamente en la capa de aplicación.

```
package com.jcaa.usersmanagement.domain.exception;

public final class EmailSenderException extends DomainException {

 private static final String DEFAULT_MESSAGE = "La notificación por correo no pudo ser enviada.";
 private static final String API_MESSAGE_WITH_DETAIL =
     "No se pudo enviar el correo a '%s' mediante la API de Brevo: %s";

 public EmailSenderException(final String message) {
   super(message);
 }

 public EmailSenderException(final String message, final Throwable cause) {
   super(message, cause);
 }

 public static EmailSenderException becauseSendFailed(final Throwable cause) {
   return new EmailSenderException(DEFAULT_MESSAGE, cause);
 }

 public static EmailSenderException becauseApiFailed(
     final String destinationEmail, final String detail) {
   return new EmailSenderException(String.format(API_MESSAGE_WITH_DETAIL, destinationEmail, detail));
 }
}
```

10. Crear las constantes (Enums) de dominio necesarias los record para ValueObjects de dominio necesarios

El record UserEmail actúa como un Value Object del dominio que encapsula la lógica de validación, normalización (conversión a minúsculas y eliminación de espacios) y formato de direcciones de correo electrónico, asegurando la integridad del dato en todo el sistema.

src/main/java/com/jcaa/usersmanagement/domain/valueobject/**UserEmail.java**

```
package com.jcaa.usersmanagement.domain.valueobject;

import com.jcaa.usersmanagement.domain.exception.InvalidUserEmailException;
import java.util.Objects;
import java.util.regex.Pattern;

public record UserEmail(String value) {

 private static final Pattern EMAIL_PATTERN =
     Pattern.compile("^[a-zA-Z0-9._%+\\-]+@[a-zA-Z0-9.\\-]+\\.[a-zA-Z]{2,}$");

 public UserEmail {
   final String normalizedValue =
       Objects.requireNonNull(value, "UserEmail cannot be null").trim().toLowerCase();
   validateNotEmpty(normalizedValue);
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
    validateFormat(normalizedValue);
    value = normalizedValue;
  }

  private static void validateNotEmpty(final String normalizedValue) {
    if (normalizedValue.isEmpty()) {
      throw InvalidUserEmailException.becauseValueIsEmpty();
    }
  }

  private static void validateFormat(final String normalizedValue) {
    if (!EMAIL_PATTERN.matcher(normalizedValue).matches()) {
      throw InvalidUserEmailException.becauseFormatIsInvalid(normalizedValue);
    }
  }

  @Override
  public String toString() {
    return value;
  }
}
```

11. Crear las clases Entidades de dominio necesarias

El modelo EmailDestinationModel representa la entidad de dominio que empaqueta los datos indispensables para efectuar un envío de correo: la dirección del destinatario, su nombre, el asunto del mensaje y el cuerpo HTML procesado.

src/main/java/com/jcaa/usersmanagement/domain/model/**EmailDestinationModel**.java

```
package com.jcaa.usersmanagement.domain.model;

import java.util.Objects;
import lombok.Value;

@Value
public class EmailDestinationModel {

  String destinationEmail;
  String destinationName;
  String subject;
  String body;

  public EmailDestinationModel(
      final String destinationEmail,
      final String destinationName,
      final String subject,
      final String body) {
    this.destinationEmail =
        validateNotBlank(destinationEmail, "El email del destinatario es requerido.");
    this.destinationName =
        validateNotBlank(destinationName, "El nombre del destinatario es requerido.");
    this.subject = validateNotBlank(subject, "El asunto es requerido.");
    this.body = validateNotBlank(body, "El cuerpo del mensaje es requerido.");
  }

  private static String validateNotBlank(final String value, final String errorMessage) {
    Objects.requireNonNull(value, errorMessage);
    if (value.trim().isEmpty()) {
      throw new IllegalArgumentException(errorMessage);
    }
    return value;
  }
}
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

12. Crear la Interfaz para el puerto de salida

La interfaz EmailSenderPort define el puerto de salida en la Arquitectura Hexagonal para el envío de correos. Su objetivo es establecer el contrato abstracto que debe implementar cualquier adaptador externo encargado de enviar mensajes.

src/main/java/com/jcaa/usersmanagement/application/port/out/**EmailSenderPort**.java

```java
package com.jcaa.usersmanagement.application.port.out;

import com.jcaa.usersmanagement.domain.model.EmailDestinationModel;

public interface EmailSenderPort {
 void send(EmailDestinationModel destination);
}
```

13. Crear una clase para la configuración de integración con el API de Brevo

El record BrevoConfig encuadra la configuración técnica requerida para conectarse con Brevo (URLs, credenciales, tiempos de espera y parámetros de reintentos/circuit breaker), incluyendo validaciones en su constructor para garantizar que las claves posean el formato REST adecuado (prefijo xkeysib-).

src/main/java/com/jcaa/usersmanagement/infrastructure/adapter/email/**BrevoConfig**.java

```java
package com.jcaa.usersmanagement.infrastructure.adapter.email;

import java.time.Duration;

public record BrevoConfig(
        String baseUrl,
        String apiKey,
        String fromAddress,
        String fromName,
        Duration connectTimeout,
        Duration readTimeout,
        int retryMaxAttempts,
        Duration retryInitialDelay,
        double retryMultiplier,
        int circuitFailureThreshold,
        int circuitSlidingWindowSize,
        int circuitMinimumCalls,
        Duration circuitOpenDuration) {

    public BrevoConfig {
        requireText(baseUrl, "brevo.base-url");
        requireText(apiKey, "brevo.api-key / BREVO_API_KEY");
        requireText(fromAddress, "brevo.from.address / BREVO_FROM_ADDRESS");
        if (!apiKey.startsWith("xkeysib-")) {
            throw new IllegalArgumentException(
                    "BREVO_API_KEY debe ser una clave API REST de Brevo con prefijo 'xkeysib-'.");
        }
    }

    private static void requireText(final String value, final String propertyName) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(
                    "La configuracion requerida '" + propertyName + "' no puede estar vacia.");
        }
    }
}
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

14. Crear una clase para configurar la crear los beans necesarios que usaremos en el adaptador

La clase BrevoSpringConfig declara los Beans de configuración en Spring Boot. Su objetivo principal es construir e inyectar el cliente HTTP síncrono RestClient configurado con las cabeceras predeterminadas de autenticación (api-key) y los tiempos de respuesta y conexión requeridos.

src/main/java/com/jcaa/usersmanagement/infrastructure/config/**BrevoSpringConfig**.java

```java
package com.jcaa.usersmanagement.infrastructure.config;

import com.jcaa.usersmanagement.infrastructure.adapter.email.BrevoConfig;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.time.Duration;
import java.util.HexFormat;

import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration(proxyBeanMethods = false)
@Slf4j
public class BrevoSpringConfig {

   @Bean
   public BrevoConfig brevoConfig(
           @Value("${brevo.base-url}") final String baseUrl,
           @Value("${brevo.local.api-key:${brevo.api-key}}") final String apiKey,
           @Value("${brevo.local.from.address:${brevo.from.address}}") final String fromAddress,
           @Value("${brevo.local.from.name:${brevo.from.name}}") final String fromName,
           @Value("${brevo.timeout.connect}") final Duration connectTimeout,
           @Value("${brevo.timeout.read}") final Duration readTimeout,
           @Value("${brevo.retry.max-attempts}") final int retryMaxAttempts,
           @Value("${brevo.retry.initial-delay}") final Duration retryInitialDelay,
           @Value("${brevo.retry.multiplier}") final double retryMultiplier,
           @Value("${brevo.circuit-breaker.failure-rate-threshold}") final int circuitFailureThreshold,
           @Value("${brevo.circuit-breaker.sliding-window-size}") final int circuitSlidingWindowSize,
           @Value("${brevo.circuit-breaker.minimum-calls}") final int circuitMinimumCalls,
           @Value("${brevo.circuit-breaker.open-duration}") final Duration circuitOpenDuration) {
       final BrevoConfig config = new BrevoConfig(
               baseUrl,
               apiKey,
               fromAddress,
               fromName,
               connectTimeout,
               readTimeout,
               retryMaxAttempts,
               retryInitialDelay,
               retryMultiplier,
               circuitFailureThreshold,
               circuitSlidingWindowSize,
               circuitMinimumCalls,
               circuitOpenDuration);
       log.info(
               "[BrevoSpringConfig] configuracion cargada. apiKeyFingerprint={} fromAddress={}",
               fingerprint(apiKey),
               fromAddress);
       return config;
   }
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ©+ⓘ+↻

```
@Bean
public RestClient brevoRestClient(final BrevoConfig config) {
    final SimpleClientHttpRequestFactory requestFactory = new SimpleClientHttpRequestFactory();
    requestFactory.setConnectTimeout(config.connectTimeout());
    requestFactory.setReadTimeout(config.readTimeout());
    return RestClient.builder()
            .baseUrl(config.baseUrl())
            .requestFactory(requestFactory)
            .defaultHeader("api-key", config.apiKey())
            .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
            .defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
            .build();
}

private static String fingerprint(final String value) {
    try {
        final byte[] digest =
                MessageDigest.getInstance("SHA-256").digest(value.getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(digest, 0, 6);
    } catch (final NoSuchAlgorithmException exception) {
        throw new IllegalStateException("SHA-256 no esta disponible", exception);
    }
  }
}
```

15. Crear una clase para realizar la petición RESTful (hacia Brevo) de forma asíncrona, en segundo plano.

La clase AsyncNotificationConfig habilita la ejecución asíncrona (@EnableAsync) en Spring Boot y configura un pool de hilos personalizado (ThreadPoolTaskExecutor). Su objetivo es procesar las notificaciones por correo en segundo plano para no bloquear el hilo principal de las solicitudes HTTP de los usuarios.

src/main/java/com/jcaa/usersmanagement/infrastructure/config/**AsyncNotificationConfig**.java

```
package com.jcaa.usersmanagement.infrastructure.config;

import java.util.concurrent.Executor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

@EnableAsync
@Configuration(proxyBeanMethods = false)
public class AsyncNotificationConfig {

 private static final int CORE_POOL_SIZE = 2;
 private static final int MAX_POOL_SIZE = 4;
 private static final int QUEUE_CAPACITY = 100;
 private static final int SHUTDOWN_WAIT_SECONDS = 30;

 @Bean(name = "notificationExecutor")
 public Executor notificationExecutor() {
   final ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
   executor.setCorePoolSize(CORE_POOL_SIZE);
   executor.setMaxPoolSize(MAX_POOL_SIZE);
   executor.setQueueCapacity(QUEUE_CAPACITY);
   executor.setThreadNamePrefix("email-notification-");
   executor.setWaitForTasksToCompleteOnShutdown(true);
   executor.setAwaitTerminationSeconds(SHUTDOWN_WAIT_SECONDS);
   executor.initialize();
   return executor;
 }
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
}
```

16. Crear una clase o record para la petición que el adaptador realizará a Brevo

El record BrevoEmailRequest mapea la estructura JSON exacta requerida por el endpoint /v3/smtp/email de la API RESTful de Brevo, estructurando los contactos del remitente, destinatarios, asunto, contenido HTML y cabeceras de idempotencia.

src/main/java/com/jcaa/usersmanagement/infrastructure/adapter/email/**BrevoEmailRequest**.java

```
package com.jcaa.usersmanagement.infrastructure.adapter.email;

import java.util.List;
import java.util.Map;

record BrevoEmailRequest(
        Contact sender,
        List<Contact> to,
        String subject,
        String htmlContent,
        Map<String, String> headers) {

    record Contact(String email, String name) {
    }
}
```

17. Crear la clase Adaptador para Brevo que implemente al puerto de salida creado en la capa de dominio

La clase BrevoEmailSenderAdapter es el adaptador de infraestructura que implementa EmailSenderPort. Su objetivo es enviar las peticiones HTTP hacia Brevo protegiendo las llamadas mediante patrones de resiliencia (Retry con Exponential Backoff y Circuit Breaker de Resilience4j) para reintentar o pausar peticiones ante fallos transitorios de red.

src/main/java/com/jcaa/usersmanagement/infrastructure/adapter/email/**BrevoEmailSenderAdapter**.java

```
package com.jcaa.usersmanagement.infrastructure.adapter.email;

import com.jcaa.usersmanagement.application.port.out.EmailSenderPort;
import com.jcaa.usersmanagement.domain.exception.EmailSenderException;
import com.jcaa.usersmanagement.domain.model.EmailDestinationModel;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
import io.github.resilience4j.core.IntervalFunction;
import io.github.resilience4j.retry.Retry;
import io.github.resilience4j.retry.RetryConfig;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.UUID;
import java.util.function.Predicate;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatusCode;
import org.springframework.stereotype.Component;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestClientResponseException;

@Slf4j
@Component
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```java
public class BrevoEmailSenderAdapter implements EmailSenderPort {

 private static final String SEND_EMAIL_PATH = "/v3/smtp/email";
 private static final String IDEMPOTENCY_HEADER = "idempotencyKey";

 private final BrevoConfig config;
 private final RestClient restClient;
 private final Retry retry;
 private final CircuitBreaker circuitBreaker;

 public BrevoEmailSenderAdapter(final BrevoConfig config, final RestClient brevoRestClient) {
   this.config = config;
   this.restClient = brevoRestClient;
   this.retry = buildRetry(config);
   this.circuitBreaker = buildCircuitBreaker(config);
   this.retry
       .getEventPublisher()
       .onRetry(
           event ->
               log.warn(
                   "[BrevoEmailSenderAdapter] reintento {} tras fallo transitorio: {}",
                   event.getNumberOfRetryAttempts(),
                   exceptionType(event.getLastThrowable())));
}

@Override
public void send(final EmailDestinationModel destination) {
  final BrevoEmailRequest request = buildRequest(destination);
  final Runnable protectedRequest =
      Retry.decorateRunnable(
          retry, CircuitBreaker.decorateRunnable(circuitBreaker, () -> sendRequest(request)));
  try {
    protectedRequest.run();
    log.info(
        "[BrevoEmailSenderAdapter] correo aceptado por Brevo para {}.",
        destination.getDestinationEmail());
  } catch (final RestClientResponseException exception) {
    throw EmailSenderException.becauseApiFailed(
        destination.getDestinationEmail(), "HTTP " + exception.getStatusCode().value());
  } catch (final CallNotPermittedException exception) {
    throw EmailSenderException.becauseApiFailed(
        destination.getDestinationEmail(), "circuit breaker abierto");
  } catch (final RestClientException exception) {
    throw EmailSenderException.becauseApiFailed(
        destination.getDestinationEmail(), exception.getClass().getSimpleName());
  }
}

private void sendRequest(final BrevoEmailRequest request) {
  restClient.post().uri(SEND_EMAIL_PATH).body(request).retrieve().toBodilessEntity();
}

private BrevoEmailRequest buildRequest(final EmailDestinationModel destination) {
  return new BrevoEmailRequest(
      new BrevoEmailRequest.Contact(config.fromAddress(), config.fromName()),
      List.of(
          new BrevoEmailRequest.Contact(
              destination.getDestinationEmail(), destination.getDestinationName())),
      destination.getSubject(),
      destination.getBody(),
      Map.of(IDEMPOTENCY_HEADER, UUID.randomUUID().toString()));
}

private static Retry buildRetry(final BrevoConfig config) {
  final IntervalFunction backoff =
      IntervalFunction.ofExponentialBackoff(
          config.retryInitialDelay().toMillis(), config.retryMultiplier());
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
  final RetryConfig retryConfig =
      RetryConfig.custom()
          .maxAttempts(config.retryMaxAttempts())
          .intervalFunction(backoff)
          .retryOnException(BrevoEmailSenderAdapter::isTransientFailure)
          .build();
  return Retry.of("brevo-email", retryConfig);
}

private static CircuitBreaker buildCircuitBreaker(final BrevoConfig config) {
  final Predicate<Throwable> transientFailure = BrevoEmailSenderAdapter::isTransientFailure;
  final CircuitBreakerConfig circuitConfig =
      CircuitBreakerConfig.custom()
          .failureRateThreshold(config.circuitFailureThreshold())
          .slidingWindowSize(config.circuitSlidingWindowSize())
          .minimumNumberOfCalls(config.circuitMinimumCalls())
          .waitDurationInOpenState(config.circuitOpenDuration())
          .recordException(transientFailure)
          .build();
  return CircuitBreaker.of("brevo-email", circuitConfig);
}

private static boolean isTransientFailure(final Throwable exception) {
  if (exception instanceof ResourceAccessException
      || exception instanceof HttpServerErrorException) {
    return true;
  }
  if (exception instanceof HttpClientErrorException clientError) {
    final HttpStatusCode status = clientError.getStatusCode();
    return status.value() == 408 || status.value() == 429;
  }
  return false;
}

private static String exceptionType(final Throwable exception) {
  return Objects.isNull(exception) ? "causa desconocida" : exception.getClass().getSimpleName();
}
}
```

18. Crear una clase en la capa de aplicación que tenga como objetivo orquestar (ejecutar o cordinar) la logica para enviar las notificaciones por email.

La clase EmailNotificationService orquesta la carga de las plantillas HTML desde los recursos, reemplaza los marcadores dinámicos con los datos del usuario y ejecuta de manera asíncrona (@Async) la llamada al puerto de envío de correos, manejando de forma limpia los posibles errores de transmisión.

```
package com.jcaa.usersmanagement.application.service;

import com.jcaa.usersmanagement.application.port.out.EmailSenderPort;
import com.jcaa.usersmanagement.domain.exception.EmailSenderException;
import com.jcaa.usersmanagement.domain.model.EmailDestinationModel;
import com.jcaa.usersmanagement.domain.model.UserModel;
import java.util.Objects;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

import java.io.IOException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.util.Map;

@Slf4j
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```java
@Service
@RequiredArgsConstructor
public class EmailNotificationService {

 private static final String SUBJECT_CREATED = "Tu cuenta ha sido creada — Gestion de Usuarios";
 private static final String SUBJECT_UPDATED  = "Tu cuenta ha sido actualizada — Gestion de Usuarios";

 private static final String TOKEN_NAME     = "name";
 private static final String TOKEN_EMAIL    = "email";
 private static final String TOKEN_PASSWORD = "password";
 private static final String TOKEN_ROLE     = "role";
 private static final String TOKEN_STATUS   = "status";

 private static final String LOG_SEND_FAILED =
     "[EmailNotificationService] correo no enviado. Causa: {}";

 private final EmailSenderPort emailSenderPort;

 @Async("notificationExecutor")
 public void notifyUserCreated(final UserModel user, final String plainPassword) {
   final String template = loadTemplate("user-created.html");
   final String body =
       renderTemplate(
           template,
           Map.of(
               TOKEN_NAME,     user.getName().value(),
               TOKEN_EMAIL,    user.getEmail().value(),
               TOKEN_PASSWORD, plainPassword,
               TOKEN_ROLE,     user.getRole().name()));
   final EmailDestinationModel destination = buildDestination(user, SUBJECT_CREATED, body);
   sendOrLog(destination);
 }

 @Async("notificationExecutor")
 public void notifyUserUpdated(final UserModel user) {
   final String template = loadTemplate("user-updated.html");
   final String body =
       renderTemplate(
           template,
           Map.of(
               TOKEN_NAME,   user.getName().value(),
               TOKEN_EMAIL,  user.getEmail().value(),
               TOKEN_ROLE,   user.getRole().name(),
               TOKEN_STATUS, user.getStatus().name()));
   final EmailDestinationModel destination = buildDestination(user, SUBJECT_UPDATED, body);
   sendOrLog(destination);
 }

 private static EmailDestinationModel buildDestination(
     final UserModel user, final String subject, final String body) {
   return new EmailDestinationModel(
       user.getEmail().value(), user.getName().value(), subject, body);
 }

 private String loadTemplate(final String templateName) {
   final String path = "/templates/" + templateName;
   try (InputStream inputStream = openResourceStream(path)) {
     if (Objects.isNull(inputStream)) {
       throw EmailSenderException.becauseSendFailed(
           new IllegalStateException("Template not found: " + path));
     }
     return new String(inputStream.readAllBytes(), StandardCharsets.UTF_8);
   } catch (final IOException ioException) {
     throw EmailSenderException.becauseSendFailed(ioException);
   }
 }
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
InputStream openResourceStream(final String path) {
  return getClass().getResourceAsStream(path);
}

private String renderTemplate(String template, final Map<String, String> values) {
  String result = template;
  for (final Map.Entry<String, String> tokenEntry : values.entrySet()) {
    final String token = "{{" + tokenEntry.getKey() + "}}";
    result = result.replace(token, tokenEntry.getValue());
  }
  return result;
}

private void sendOrLog(final EmailDestinationModel destination) {
  try {
    emailSenderPort.send(destination);
  } catch (final EmailSenderException senderException) {
    log.warn(LOG_SEND_FAILED, senderException.getMessage(), senderException);
    throw senderException;
  }
}
}
```

19. En la clase de servicio. (Capa de aplicación).

Los que gestiona (orquesta o coordina) la lógica para crear usuarios debemos inyectar una instancia del servicio que orquesta el envío de las notificaciones, de esta manera podremos enviar una notificación a cada usuario que se ha creado.

El servicio UpdateUserService coordina el caso de uso de actualización de usuarios en la capa de aplicación. Integra la invocación a EmailNotificationService para disparar la notificación por correo al usuario justo después de haber guardado exitosamente los cambios en la base de datos.

```
package com.jcaa.usersmanagement.application.service;

import com.jcaa.usersmanagement.application.port.in.UpdateUserUseCase;
import com.jcaa.usersmanagement.application.port.out.GetUserByEmailPort;
import com.jcaa.usersmanagement.application.port.out.GetUserByIdPort;
import com.jcaa.usersmanagement.application.port.out.UpdateUserPort;
import com.jcaa.usersmanagement.application.service.dto.command.UpdateUserCommand;
import com.jcaa.usersmanagement.application.service.mapper.UserApplicationMapper;
import com.jcaa.usersmanagement.domain.exception.UserAlreadyExistsException;
import com.jcaa.usersmanagement.domain.exception.UserNotFoundException;
import com.jcaa.usersmanagement.domain.model.UserModel;
import com.jcaa.usersmanagement.domain.valueobject.UserEmail;
import com.jcaa.usersmanagement.domain.valueobject.UserId;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Validator;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.Set;

@Service
@RequiredArgsConstructor
public class UpdateUserService implements UpdateUserUseCase {

 private final UpdateUserPort updateUserPort;
 private final GetUserByIdPort getUserByIdPort;
 private final GetUserByEmailPort getUserByEmailPort;
 private final EmailNotificationService emailNotificationService;
 private final Validator validator;

 @Override
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** cc + by + sa

```
public UserModel execute(final UpdateUserCommand command) {
  validateCommand(command);

  final UserId userId = new UserId(command.id());
  final UserModel current = findExistingUserOrFail(userId);
  final UserEmail newEmail = new UserEmail(command.email());

  ensureEmailIsNotTakenByAnotherUser(newEmail, userId);

  final UserModel userToUpdate =
      UserApplicationMapper.fromUpdateCommandToModel(command, current.getPassword());
  final UserModel updatedUser = updateUserPort.update(userToUpdate);

  emailNotificationService.notifyUserUpdated(updatedUser);

  return updatedUser;
}

private void validateCommand(final UpdateUserCommand command) {
  final Set<ConstraintViolation<UpdateUserCommand>> violations = validator.validate(command);
  if (!violations.isEmpty()) {
    throw new ConstraintViolationException(violations);
  }
}

private UserModel findExistingUserOrFail(final UserId userId) {
  return getUserByIdPort
      .getById(userId)
      .orElseThrow(() -> UserNotFoundException.becauseIdWasNotFound(userId.value()));
}

private void ensureEmailIsNotTakenByAnotherUser(final UserEmail newEmail, final UserId ownerId) {
  getUserByEmailPort
      .getByEmail(newEmail)
      .ifPresent(
          found -> {
            if (!found.getId().equals(ownerId)) {
              throw UserAlreadyExistsException.becauseEmailAlreadyExists(newEmail.value());
            }
          });
}
}
```

20. Inyectar una instancia de servicio de notificacion tambien en la clase de servicio que orquesta la logica para actualizar usuarios, esto nos permite notificar a cada usuario cuando se cambien sus datos.

```
package com.jcaa.usersmanagement.application.service;

import com.jcaa.usersmanagement.application.port.in.UpdateUserUseCase;
import com.jcaa.usersmanagement.application.port.out.GetUserByEmailPort;
import com.jcaa.usersmanagement.application.port.out.GetUserByIdPort;
import com.jcaa.usersmanagement.application.port.out.UpdateUserPort;
import com.jcaa.usersmanagement.application.service.dto.command.UpdateUserCommand;
import com.jcaa.usersmanagement.application.service.mapper.UserApplicationMapper;
import com.jcaa.usersmanagement.domain.exception.UserAlreadyExistsException;
import com.jcaa.usersmanagement.domain.exception.UserNotFoundException;
import com.jcaa.usersmanagement.domain.model.UserModel;
import com.jcaa.usersmanagement.domain.valueobject.UserEmail;
import com.jcaa.usersmanagement.domain.valueobject.UserId;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Validator;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
```

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ©+ⓘ+↻

```
import java.util.Set;

@Service
@RequiredArgsConstructor
public class UpdateUserService implements UpdateUserUseCase {

 private final UpdateUserPort updateUserPort;
 private final GetUserByIdPort getUserByIdPort;
 private final GetUserByEmailPort getUserByEmailPort;
 private final EmailNotificationService emailNotificationService;
 private final Validator validator;

 @Override
 public UserModel execute(final UpdateUserCommand command) {
   validateCommand(command);

   final UserId userId = new UserId(command.id());
   final UserModel current = findExistingUserOrFail(userId);
   final UserEmail newEmail = new UserEmail(command.email());

   ensureEmailIsNotTakenByAnotherUser(newEmail, userId);

   final UserModel userToUpdate =
       UserApplicationMapper.fromUpdateCommandToModel(command, current.getPassword());
   final UserModel updatedUser = updateUserPort.update(userToUpdate);

   emailNotificationService.notifyUserUpdated(updatedUser);

   return updatedUser;
 }

 private void validateCommand(final UpdateUserCommand command) {
   final Set<ConstraintViolation<UpdateUserCommand>> violations = validator.validate(command);
   if (!violations.isEmpty()) {
     throw new ConstraintViolationException(violations);
   }
 }

 private UserModel findExistingUserOrFail(final UserId userId) {
   return getUserByIdPort
       .getById(userId)
       .orElseThrow(() -> UserNotFoundException.becauseIdWasNotFound(userId.value()));
 }

 private void ensureEmailIsNotTakenByAnotherUser(final UserEmail newEmail, final UserId ownerId) {
   getUserByEmailPort
       .getByEmail(newEmail)
       .ifPresent(
           found -> {
             if (!found.getId().equals(ownerId)) {
               throw UserAlreadyExistsException.becauseEmailAlreadyExists(newEmail.value());
             }
           });
 }
}
```

21. Crear el archivo de propiedades para las test unitarias.

El archivo application.properties ubicado en la carpeta de pruebas (src/test/resources) provee valores ficticios de configuración para aislar los tests unitarios e integrar un contexto de ejecución seguro sin necesidad de conectarse a servicios reales de producción.

src/test/resources/**application.properties**

CC-BY.cc **Instructor**: John Carlos Arrieta Arrieta. arrietajohn@gmail.com - **Licencia Creative Common - By Author:** ⓒ+ⓘ+↻

```
# ============================================
# Test configuration (fictitious values only)
# ============================================

# Evita levantar contexto web en tests Mockito puros (@ExtendWith(MockitoExtension.class))
spring.main.web-application-type=none
spring.autoconfigure.exclude[0]=org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAut
oConfiguration

db.host=localhost
db.port=3306
db.name=test_db
db.username=test_user
db.password=test_pass

# ============================================
# Brevo test configuration
# ============================================
brevo.base-url=http://localhost
brevo.api-key=xkeysib-test-api-key
brevo.from.address=test@test.local
brevo.from.name=Test User
brevo.timeout.connect=100ms
brevo.timeout.read=100ms
brevo.retry.max-attempts=1
brevo.retry.initial-delay=1ms
brevo.retry.multiplier=2.0
brevo.circuit-breaker.failure-rate-threshold=50
brevo.circuit-breaker.sliding-window-size=10
brevo.circuit-breaker.minimum-calls=5
brevo.circuit-breaker.open-duration=1s

jwt.secret=dGVzdC1vbmx5LWp3dC1zZWNyZXQtd2l0aC1hdC1sZWFzdC0zMi1ieXRlcw==
jwt.expiration-seconds=900
```

22. Crear las Test Unitarias requeridas para alcanzar una cobertura mínimo del 90%

Compilar:

```
PS C:\Users\pc\Documents\UNICOLOMBO\electiva2\users-management-spring-boot-hexagonal-ddd> mvn clean install -U
```

Ejecutar y probar con Postman o Boomerang o Insomnia.